

UNIVERSITATEA BABEȘ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA Informatică

LUCRARE DE LICENȚĂ

**Sentinel: Sistem de Detectare și
Răspuns la Intruziuni bazat pe eBPF
cu Scoring Anomalistic Augmentat
prin Învățare Automată**

Conducător științific
[Grad, titlu și nume coordonator]

Absolvent
[Nume student]

ABSTRACT

This thesis presents the design and implementation of Sentinel, an eBPF-based Host Intrusion Detection and Response System (HIDS) for the Linux platform. Sentinel attaches kprobes to 21 security-relevant system calls, collects events through a 64 MiB ring buffer and processes them in a Go user-space daemon. A three-layer hybrid scoring engine assigns risk scores: static per-syscall weights calibrated on empirical malware data, a sliding-window burst detector, and an optional Isolation Forest ML scorer served through a REST API. Scores are mapped to six severity states (OK / WATCH / WARN / THROTTLE / ISOLATE / FREEZE) and a quarantine manager applies proportional containment actions via Linux cgroup v2 and nftables — CPU throttling at 10%, memory capping at 256 MiB, network isolation by cgroup inode, and full process freezing via SIGSTOP and `cgroup.freeze`. The system is validated against a reproducible suite of 21 attack scenarios covering kernel rootkits, privilege escalation, process injection, network exfiltration, persistence and fileless execution. Measured containment latency is below 5 ms; eBPF overhead is approximately 1.2% CPU on a kernel-build workload. Original contributions include the adaptive weighted scoring mechanism, the cgroup-inode-based selective network isolation, the PID-reuse-safe burst key, and the self-contained attack test suite.

Cuprins

1	Introducere	1
1.1	Contextul și motivația	1
1.2	Enunțul problemei	2
1.3	Obiectivele lucrării	2
1.4	Contribuții originale	3
1.5	Structura lucrării	3
2	Fundamente teoretice și contextul actual	5
2.1	Modelul de securitate Linux	5
2.1.1	Apeluri de sistem și frontiera kernel-utilizator	5
2.1.2	Linux Security Modules	6
2.1.3	Namespaces și grupuri de control (cgroups)	7
2.2	Extended Berkeley Packet Filter (eBPF)	7
2.2.1	Evoluție de la BPF clasic la eBPF	7
2.2.2	Maps eBPF și mecanismul ring buffer	8
2.2.3	Kprobes: instrumentarea dinamică a kernel-ului	8
2.3	Sisteme de detectare a intruziunilor	9
2.3.1	Taxonomie și clasificare	9
2.3.2	Detecție bazată pe anomalii versus semnături	9
2.4	Algoritmul Isolation Forest	10
2.5	Instrumente existente și lucrări corelate	10
2.6	Peisajul amenințărilor în ecosistemul Linux și Cadrul MITRE ATT&CK	11
2.6.1	Rootkit-uri și persistența la nivel de nucleu	12
2.6.2	Escaladarea privilegiilor	12
2.6.3	Evaziunea defensivă și execuția <i>fileless</i>	12
2.6.4	Injectarea de procese	13
2.6.5	Accesul la credențiale și comanda-și-control	13
3	Analiza și Proiectarea Sistemului Sentinel	14
3.1	Cerințe funcționale și nefuncționale	14
3.2	Arhitectura sistemului	15

3.3	Proiectarea programului eBPF	16
3.4	Proiectarea motorului de scoring	17
3.4.1	Stratul 1: Evaluarea statică a apelurilor de sistem	17
3.4.2	Stratul 2: Analiza comportamentală a activității în rafală (Burst Detection)	17
3.4.3	Stratul 3: Analiza contextuală bazată pe căile fișierelor	18
3.4.4	Integrarea scorerului ML	18
3.5	Mașina de stări pentru carantinare	19
3.6	Schema bazei de date	20
4	Implementare și Testare	21
4.1	Mediul de dezvoltare și testare	21
4.2	Implementarea programului eBPF	22
4.3	Implementarea daemonului Go	24
4.4	Implementarea scorerului ML	27
4.5	Implementarea managerului de carantinare	29
4.6	Suita de teste de atac	32
4.6.1	Metodologia de testare	32
4.6.2	Scenariile de bază (REP-01 – REP-12)	33
4.6.3	Scenariile avansate (REP-13 – REP-21)	34
4.7	Clasificarea și descrierea teoretică a suitei de atacuri	35
4.7.1	Rootkit-uri și persistență la nivel de nucleu (Module LKM)	35
4.7.2	Injectarea de procese și manipularea memoriei	36
4.7.3	Evaziunea defensivă: execuție <i>fileless</i> și memorie RWX	38
4.7.4	Anomalii de rețea, exfiltrare și C2	39
4.7.5	Escaladarea privilegiilor și evadarea din medii izolate	40
4.7.6	Manipularea fișierelor, persistență și recoltare	41
4.7.7	Anomalii de execuție și epuizarea resurselor	43
4.8	Considerente de performanță	43
5	Concluzii	45
	Contribuții originale	46
	Limitări	46
	Direcții de cercetare viitoare	47
	Bibliografie	49

Capitolul 1

Introducere

Securitatea sistemelor de operare bazate pe Linux a dobândit o importanță tot mai mare pe măsură ce aceste sisteme au ajuns să susțină infrastructuri critice: servere web, platforme de containere, sisteme de calcul de înaltă performanță și dispozitive IoT. Creșterea numărului de atacuri direcționate împotriva nucleului Linux — rootkit-uri, tehnici de injecție de procese, escaladare de privilegii și execuție fără fișier pe disc — impune soluții de detecție capabile să opereze la nivelul cel mai profund al stivei software, fără a introduce latențe inacceptabile sau vulnerabilități suplimentare.

Lucrarea de față descrie proiectarea și implementarea sistemului *Sentinel*, un sistem de detectare și răspuns la intruziuni bazat pe tehnologia *extended Berkeley Packet Filter* (eBPF). Sistemul interceptează apeluri de sistem la nivel de kernel prin mecanismul *kprobe*, calculează un scor de risc compus pentru fiecare eveniment detectat și aplică automat acțiuni graduale de containment al proceselor suspectate. *Sentinel* funcționează fără a modifica codul kernel-ului și fără a fi necesară repornirea sistemului — proprietăți esențiale pentru medii de producție.

1.1 Contextul și motivația

Instrumentele tradiționale de monitorizare a securității, cum ar fi demonul de audit Linux *auditd* [Lin24] sau sistemul Snort bazat pe semnături de rețea [Roe99], au demonstrat limitări documentate în fața atacurilor sofisticate care operează exclusiv în memorie sau care manipulează structurile interne ale kernel-ului [CGFB18]. Abordările bazate pe analiza jurnalelor suferă de o latență inerentă: evenimentul este produs, jurnalizat, scris pe disc și abia apoi analizat, permițând atacatorilor o fereastră de acțiune neobservată de ordinul secundelor sau minutelor.

eBPF oferă o paradigmă diferită. Programele eBPF se execută într-un sandbox verificat formal al kernel-ului Linux, atașate la puncte de instrumentare (*kprobes*,

tracepoints, uprobes) cu o suprasarcină de ordinul zecilor de nanosecunde per eveniment [Gre19]. Aceasta permite vizibilitate completă, în timp real, asupra comportamentului oricărui proces, fără modificarea kernel-ului sau instalarea de module de kernel suplimentare.

Complementar, metodele de detecție a anomaliilor bazate pe învățare automată — în special algoritmul Isolation Forest [LTZ08] — s-au dovedit eficiente pentru identificarea comportamentelor atipice în spații de caracteristici de dimensionalitate medie, inclusiv în contextul securității informatice [BG16]. Integrarea unui scorer ML cu un agent de detecție bazat pe eBPF formează o arhitectură hibridă care combină viteza și determinismul regulilor statice cu capacitatea de generalizare a modelelor statistice.

1.2 Enunțul problemei

Problema adresată de această lucrare poate fi formulată astfel: *cum poate fi proiectat un sistem de detectare și răspuns la intruziuni pentru Linux care (a) operează la nivel de kernel fără a-l modifica, (b) clasifică evenimentele de securitate în timp real pe baza unui scor de risc compus și (c) aplică automat acțiuni de containment proporționale cu severitatea detectată, cu o latență de răspuns sub cinci milisecunde?*

Validarea empirică se realizează printr-o suită de 21 de scenarii de atac reproductibile, ce acoperă principalele categorii de amenințări documentate în literatura de specialitate: rootkit-uri de kernel, escaladare de privilegii, injecție de procese, exfiltrare de date în rețea, persistență și execuție de cod fără fișier pe disc.

1.3 Obiectivele lucrării

Obiectivele principale ale lucrării sunt:

- Proiectarea și implementarea unui program eBPF pentru interceptia selectivă a apelurilor de sistem relevante din perspectiva securității, cu suprasarcină minimă asupra sistemului.
- Construirea unui motor de scoring hibrid care combină ponderi statice per syscall, detecție a activității în rafale (burst detection) și un scorer ML bazat pe Isolation Forest, care poate fi antrenat cu date actualizate fără întreruperi.
- Implementarea unui agent de carantinare care aplică acțiuni de izolare graduale — limitare CPU, izolare de rețea, înghețarea procesului — prin mecanismele cgroup v2 și nftables.

- Validarea întregului sistem printr-o suită de teste automatizate ce rulează 21 de scenarii de atac într-un mediu izolat bazat pe mașini virtuale Vagrant.

1.4 Contribuții originale

Contribuțiile originale ale lucrării sunt:

1. **Un motor de scoring cu trei straturi:** ponderi statice ponderate per syscall, detectare a activității în rafale cu fereastră glisantă și scorer Isolation Forest care poate fi antrenat cu date actualizate prin API REST, cu un mecanism de combinare ponderată a scorurilor care previne degradarea excesivă a scorului bazal.
2. **O mașină de stări cu șase nivele** (OK / WATCH / WARN / THROTTLE / ISOLATE / FREEZE) care decuplează detecția de răspuns, permițând acțiuni proporționale cu severitatea detectată și auto-dezghețarea proceselor înghețate după un interval configurabil.
3. **O suită de 21 de scenarii de atac reproductibile**, distribuite ca artefacte de testare, care acoperă vectori de atac documentați în literatura de specialitate — de la rootkit-uri tip Diamorphine până la tehnici de execuție de cod fără fișier și rootkit-uri eBPF de tip TripleCross.
4. **Mecanismul de izolare de rețea bazat pe cgroup inode:** procesele carantinate sunt blocate în rețea prin reguli nftables care filtrează traficul după inodeul cgroup-ului, fără a afecta restul proceselor de pe sistem.

1.5 Structura lucrării

Capitolul 2 prezintă fundamentele teoretice: modelul de securitate Linux (apeluri de sistem, Linux Security Modules, namespaces și cgroups), arhitectura eBPF, clasificarea și descrierea sistemelor de detectare a intruziunilor, bazele algoritmului Isolation Forest și o analiză comparativă a instrumentelor existente. Capitolul 3 descrie analiza cerințelor și proiectarea detaliată a componentelor sistemului, inclusiv motorul de scoring și mașina de stări pentru carantinare. Capitolul 4 acoperă implementarea concretă și rezultatele testării cu suita de 21 de atacuri. Capitolul 5 sintetizează concluziile și direcțiile de cercetare viitoare.

Notă privind utilizarea instrumentelor de inteligență artificială: la redactarea unor secțiuni ale acestei lucrări a fost utilizat asistentul Claude (Anthropic) pentru reformularea și îmbunătățirea lizibilității paragrafelor tehnice. De asemenea, asistentul Claude (Anthropic) a mai fost folosit în formatarea codului și în implementarea

unor secțiuni de cod generice. Întregul conținut tehnic, implementarea software, excluzând secțiunile menționate anterior și rezultatele experimentale aparțin exclusiv autorului lucrării.

Capitolul 2

Fundamente teoretice și contextul actual

Prezentul capitol stabilește fundamentul teoretic necesar înțelegerii și proiectării arhitecturii sistemului Sentinel. Sunt analizate mecanismele intrinseci de securitate ale nucleului Linux, inovațiile aduse de tehnologia eBPF, taxonomia sistemelor de detectare a intruziunilor, aparatul matematic al algoritmului Isolation Forest, precum și o analiză comparativă a soluțiilor consacrate în domeniu.

2.1 Modelul de securitate Linux

2.1.1 Apeluri de sistem și frontiera kernel-utilizator

Granița fundamentală de izolare între codul cu privilegii reduse (spațiul utilizator / user-space) și codul cu privilegii absolute (nucleul / kernel-space) în sistemele Linux este definită de interfața apelurilor de sistem (syscalls). Orice acțiune care necesită interacțiunea cu resursele hardware sau logice ale sistemului — instanțierea de conexiuni (`socket`), crearea de procese, manipularea sistemului de fișiere, alterarea privilegiilor — trebuie să traverseze această interfață printr-o tranziție de context strict controlată [Ker10].

Din perspectiva securității cibernetice, monitorizarea apelurilor de sistem oferă o vizibilitate exhaustivă și imposibil de eludat asupra comportamentului real al unui proces: o rutină malițioasă care încearcă să încarce un modul de kernel va invoca inevitabil `finit_module` (syscall 313), o tentativă de escaladare a privilegiilor va apela `setuid` (syscall 105) sau `capset` (syscall 126), în timp ce exfiltrarea datelor va fi tradusă prin generarea de apeluri `socket` și `connect` (syscall-urile 41, respectiv 42) [Lov10].

Kernel-ul Linux x86_64 expune peste 300 de apeluri de sistem, dar numai un subsamblu specific prezintă interes direct din perspectiva securității. Sistemul Sen-

tinel monitorizează 21 de apeluri de sistem, selectate pe baza incidenței lor empirice în anatomia malware-ului pentru Linux [CGFB18], acoperind categoriile: execuție de procese, modificare de privilegii, operații de memorie, activitate de rețea, operații pe fișiere sensibile și management de module de kernel. Tabelul 2.1 prezintă o selecție cu greutatea de risc corespunzătoare.

Syscall	Nr.	Categorie alertă	Pondere
finit_module	313	HiddenPID	95
init_module	175	HiddenPID	95
delete_module	176	HiddenPID	85
ptrace	101	ProcessInjection	75
setuid	105	PrivilegeEscalation	70
setgid	106	PrivilegeEscalation	70
capset	126	PrivilegeEscalation	70
execve	59	ParentChildAnomaly	50
mprotect	10	MemoryInjection	45
mmap	9	MemoryInjection	40
connect	42	NetworkAnomaly	15
socket	41	NetworkAnomaly	10

Tabela 2.1: Selecție de apeluri de sistem monitorizate de Sentinel cu greutatea de risc asociate

2.1.2 Linux Security Modules

Cadrul Linux Security Modules (LSM), teoretizat și implementat de Wright et al. [WCS⁺02], pune la dispoziție o arhitectură bazată pe cârlige de interceptare (hooks) plasate strategic în punctele decizionale ale nucleului. Această infrastructură permite modulelor de securitate (ex. SELinux, AppArmor) să impună politici stricte de tip Mandatory Access Control (MAC).

O limitare inherentă a modulelor LSM tradiționale este natura lor declarativă și statică, nefiind concepute pentru analiza comportamentală dinamică a secvențelor de execuție. Din acest motiv, un sistem de detectare a anomaliilor comportamentale, precum Sentinel, reprezintă o completare necesară și ortogonală față de mecanismele LSM. Deși eBPF permite atașarea programelor la punctele de interceptare LSM (prin `BPF_PROG_TYPE_LSM`, funcționalitate introdusă în Linux 5.7), Sentinel utilizează tehnologia kprobes pentru a asigura o compatibilitate retroactivă mai largă pe nucleele ≥ 5.4 .

2.1.3 Namespaces și grupuri de control (cgroups)

Paradigmele Namespaces (PID, net, mount, user, UTS, IPC) asigură izolarea stării globale a kernel-ului în contexte virtualizate pentru grupuri distincte de procese, reprezentând fundamentul tehnologiilor moderne de containerizare [Ker10]. În paralel, mecanismul Control Groups v2 (cgroup v2), fundamentat inițial de Menage [Men07] și standardizat în nucleul 4.5, gestionează alocarea, limitarea și monitorizarea resurselor fizice (CPU, memorie, I/O bloc) pentru ierarhii de procese.

Spre deosebire de iterația anterioară (cgroup v1), arhitectura v2 propune o ierarhie unificată și un model de delegare mai coerent, eliminând ambiguitățile de configurare care apăreau la apartenența simultană a unui proces la sub-arbori de control diferiți. Arhitectura de izolare a sistemului Sentinel impune la pornire prezența controllerelor `cpu` și `memory` în `/sys/fs/cgroup/cgroup.controllers`, condiție satisfăcută nativ pe distribuțiile Debian 12 și Ubuntu 22.04+.

2.2 Extended Berkeley Packet Filter (eBPF)

2.2.1 Evoluție de la BPF clasic la eBPF

Berkeley Packet Filter (BPF) a fost conceptualizat de McCanne și Jacobson în 1993, vizând dezvoltarea unei arhitecturi extrem de eficiente pentru capturarea și filtrarea traficului de rețea, operând printr-o mașină virtuală minimalistă pe 32 de biți [MJ93]. Schimbarea de paradigmă pe care a propus-o BPF a fost executarea logicii de filtrare direct în kernel, evaluând pachetele la sursă pentru a preveni transferul inutil de date peste frontiera privilegiată.

Începând cu Linux 3.18 (2014), această arhitectură a suferit o extensie masivă sub acronimul eBPF: lărgirea registrelor la 64 de biți, îmbogățirea setului de instrucțiuni, introducerea structurilor de date partajate (maps) și capacitatea de atașare asincronă la aproape orice funcție a sistemului de operare [Ric23]. O inovație critică o reprezintă Verificatorul Formal (The Verifier), un mecanism care analizează codul înainte de execuție pentru a garanta finalizarea acestuia (lipsa buclelor infinite) și respectarea strictă a limitelor de memorie accesată. Compilatorul JIT (Just-In-Time) convertește ulterior bytecode-ul validat în cod mașină nativ, reducând la minimum costul de performanță (overhead).

Ecosistemul eBPF a evoluat rapid: bibliotecile `libbpf` și `cilium/ebpf` [Cil24] oferă o interfață Go/C de nivel înalt pentru încărcarea și gestionarea programelor eBPF, iar CO-RE (Compile Once, Run Everywhere) rezolvă problema portabilității între versiuni de kernel prin relocări BTF.

2.2.2 Maps eBPF și mecanismul ring buffer

Maps eBPF sunt structuri de stocare persistente, expuse asincron atât spațiului kernel, cât și celui utilizator. Arhitectura Sentinel implementează trei structuri fundamentale:

- `BPF_MAP_TYPE_HASH`: tabelă de dispersie alocată pentru păstrarea exclude-rilor (`pid_blacklist`).
- `BPF_MAP_TYPE_PERCPU_ARRAY`: vector instanțiat per-CPU pentru o contorizare atomică, fără blocaje (lock-free), a evenimentelor suprimate (`drop_count`).
- `BPF_MAP_TYPE_RINGBUF`: coadă circulară de mare viteză, dimensionată la 64 MiB, utilizată ca bus principal de transmitere a telemetriei.

Ring buffer-ul (introdus în kernel 5.8) oferă avantaje față de perf buffer-ul precedent: elimină o copiere suplimentară de date, suportă rezervarea atomică a spațiului urmată de scriere directă și notificarea consumatorului printr-un descriptor de fișier `epollable`. Gregg [Gre19] documentează că ring buffer-ul reduce latența de notificare cu 20–40% față de perf buffer în scenariile de evenimente cu frecvență ridicată.

2.2.3 Kprobes: instrumentarea dinamică a kernel-ului

Mecanismul kprobes (Kernel Probes) facilitează instrumentarea la cald a codului nucleului, permițând injectarea rutinelor de tratare (handlers) la nivelul aproape oricărei instrucțiuni asamblor, fără a necesita recompilarea sistemului [CRKH05]. La activarea unei sonde kprobe, instrucțiunea originală este înlocuită cu o întrerupere software (INT3 pe x86). Când execuția lovește acest punct, controlul este deviat către mașina virtuală eBPF, după care instrucțiunea originală este reluată. Această deviere induce o latență microscopică, de ordinul a 100–300 nanosecunde per apel [VCP⁺20], neglijabilă în raport cu durata unui syscall tipic (microsecunde–milisecunde).

Sentinelul atașează kprobe-uri pe funcțiile `__x64_sys_{name}` corespunzătoare celor 21 de syscall-uri monitorizate. Atașarea este dinamică: la pornire, daemonul iterează peste toate programele din colecția eBPF cu prefix `kp_` și le atașează la funcțiile kernel corespunzătoare, fără a fi necesară enumerarea lor explicită în codul Go.

2.3 Sisteme de detectare a intruziunilor

2.3.1 Taxonomie și clasificare

Conceptul monitorizării securității a fost formalizat inițial de Anderson (1980) ca o metodă de prelucrare a jurnalelor de audit în scopul detectării utilizării neautorizate a sistemelor informatice [And80]. Ulterior, Denning (1987) a publicat fundamentul teoretic al sistemelor IDS, propunând un model independent de sistem capabil să evalueze statistic deviațiile de la comportamentul istoric normal [Den87].

Taxonomia propusă de Axelsson (2000) [Axe00] oferă un cadru de clasificare în care Sentinel se raportează astfel:

- *Sursa datelor*: HIDS (Host-based) versus NIDS (Network-based). Sentinel este exclusiv un sistem HIDS, rezoluția sa fiind limitată la apelurile de sistem ale mașinii gazdă.
- *Paradigma de detecție*: Detecție bazată pe semnături (misuse detection) versus anomalii. Sistemul de față abordează problema hibrid, combinând un set de euristici statice (semnături) cu un model nesupervizat pentru anomalii (Isolation Forest).
- *Frecvența analizei*: detecție continuă versus periodică. Sentinel operează în timp real, cu latența limitată de viteza ring buffer-ului.
- *Tipul de răspuns*: alertare pasivă versus limitare activă (containment). Sentinel oferă un răspuns activ, autonom și gradual.

2.3.2 Detecție bazată pe anomalii versus semnături

Abordarea bazată pe semnături excelează prin rata infimă a rezultatelor fals pozitive pentru atacurile cunoscute, însă suferă din cauza ineficienței totale în fața atacurilor neclasificate (zero-day) [GTDVMFV09]. Alternativa, detecția anomaliilor, profilează starea de normalitate a sistemului; deși superioară în identificarea vectorilor de atac noi, aceasta este penalizată de un volum mai mare de rezultate fals pozitive [CBK09].

În stabilirea algoritmului adecvat, studiul lui Goldstein și Uchida [GU16] demonstrează performanța net superioară a modelului Isolation Forest pe distribuții de date unde anomaliile reprezintă un procent extrem de redus ($< 5\%$), modelând perfect realitatea evenimentelor de securitate. Această alegere este validată și de lucrarea de sinteză elaborată de Buczak și Guven [BG16], care evidențiază eficiența metodelor de ansamblu (ensemble methods) pe spații de date cu dimensionalitate variabilă.

2.4 Algoritmul Isolation Forest

Isolation Forest (iForest), formulat de Liu, Ting și Zhou [LTZ08, LTZ12], schimbă paradigma clasică a detecției anomaliilor: în loc să profileze marea masă a datelor normale, algoritmul încearcă să izoleze explicit valorile aberante. Postulatul principal este că anomaliile reprezintă instanțe rare și divergente structural; prin urmare, o serie de partiționări aleatorii (random splits) va izola un punct anormal mult mai aproape de rădăcina unui arbore de decizie comparativ cu un punct benign.

Formal, dat fiind un set de antrenament $X = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$, construirea unui arbore de izolare presupune selecția aleatoare a unui atribut $q \in \{1, \dots, d\}$ și partiționarea spațiului pe un punct p ales dintr-o distribuție uniformă în intervalul $[\min_q(X), \max_q(X)]$. Evaluarea (scorul de anomalie) pentru un punct x este obținută prin relația:

$$s(x, n) = 2^{-\frac{E[h(x)]}{c(n)}} \quad (2.1)$$

unde $E[h(x)]$ reprezintă valoarea așteptată (adâncimea medie) a lungimii căii către punctul x generată pe ansamblul de arbori, iar $c(n) = 2H(n-1) - \frac{2(n-1)}{n}$ este adâncimea medie asimptotică într-un arbore binar de căutare nereușită, cu H funcția armonică [LTZ12]. Scorul variază în intervalul $(0, 1)$: valori aproape de 1 indică anomalii puternice, valori aproape de 0.5 indică puncte normale sau ambigue.

Deoarece scorul brut emis de model (valoarea funcției de decizie `decision_function`, negativă pentru anomalii) necesită o conversie pe scala operațională $[0, 100]$, Sentinel aplică o transformare sigmoidă:

$$\text{threat_score} = \frac{100}{1 + e^{-25 \cdot (-\text{raw_score})}} \quad (2.2)$$

Factorul de scalare 25 a fost ales empiric pentru a produce o separare clară între punctele normale (`threat_score` < 30) și anomalii moderate (`threat_score` ∈ [50, 70]), menținând în același timp sensibilitate pentru valorile extreme.

Complexitatea antrenării este $\mathcal{O}(t \cdot \psi \cdot \log \psi)$, cu t numărul de arbori și ψ dimensiunea sub-eșantionării (subsampling), în timp ce faza de inferență se execută în timp $\mathcal{O}(t \cdot \log \psi)$ per eveniment — acceptabilă pentru bugetul de latență impus de constrângerile decizionale în timp real (RN2).

2.5 Instrumente existente și lucrări corelate

Falco [Clo24], proiect incubat de CNCF și derivat din Sysdig, este cel mai apropiat echivalent open-source al Sentinelului: utilizează eBPF sau un driver de kernel pentru monitorizarea syscall-urilor și permite definirea de reguli de alertă în

YAML. Spre deosebire de Sentinel, Falco nu implementează un scorer ML integrat, nu aplică acțiuni automate de containment (răspunsul este delegat unor sisteme externe precum Falcosidekick) și nu menține o bază de date locală a evenimentelor pentru analiza retrospectivă.

auditd [Lin24] este componenta de audit a kernel-ului Linux, standardizată și prezentă în toate distribuțiile majore. Jurnalizează apeluri de sistem pe baza unor reguli configurate static. Din pricina prelucrării asincrone și a interacțiunii constante cu sistemul de fișiere pentru stocarea jurnalelor, auditd induce o latență incompatibilă cu acțiunile de izolare în timp real, limitându-se la rolul de analiză retrospectivă (forensics).

Virtual Machine Introspection (VMI): Metodologia teoretizată de Garfinkel și Rosenblum (2003) [GR03] propune securizarea prin mutarea monitorului în hipervizor, complet izolat de sistemul de operare monitorizat. Implementări precum DRAKVUF [LMP⁺14] utilizează extensiile de virtualizare (Xen) pentru a analiza memoria mașinii virtuale din exterior. Limita severă a acestei abordări este dependența de virtualizarea hardware, fiind incompatibilă cu mediile bare-metal sau infrastructurile cloud moderne bazate pe containere.

kGuard [KPK12] abordează o problemă complementară — protecția kernel-ului împotriva atacurilor de tip return-to-user — prin instrumentarea statică a codului kernel la compilare. Aceste abordări de securitate prin compilare sunt ortogonale față de Sentinel, care se concentrează pe detectarea comportamentului la runtime.

O constatare empirică importantă din literatura de specialitate provine din Cozzi et al. [CGFB18]: 70% din eșantioanele de malware Linux analizate utilizează socket-uri de rețea, 30% apelează `ptrace` sau mapează pagini cu drepturi de execuție, iar 18% utilizează module de kernel. Aceste statistici au constituit criteriul de selecție și de ponderare a syscall-urilor monitorizate de Sentinel.

2.6 Peisajul amenințărilor în ecosistemul Linux și Cadrul MITRE ATT&CK

Evaluarea oricărui sistem modern de detectare a intruziunilor necesită raportarea riguroasă la un cadru standardizat de amenințări. În ecosistemul Linux, vectorii de atac au evoluat de la simple scripturi de exploatare la arhitecturi complexe, mapate exhaustiv în cadrul MITRE ATT&CK [MIT24]. Evaluarea eficacității sistemului Sentinel se fundamentează pe o analiză teoretică a 21 de scenarii de atac care acoperă tactici critice: persistență, escaladarea privilegiilor, evaziune defensivă, acces la credențiale și execuție.

În continuare sunt detaliate principalele categorii teoretice de amenințări care

justifică arhitectura de monitorizare a sistemului:

2.6.1 Rootkit-uri și persistența la nivel de nucleu

Compromiterea nucleului Linux s-a realizat istoric prin rootkit-uri de tip LKM (*Loadable Kernel Module*), care vizează alterarea fluxului de execuție din spațiul privilegiat. Un etalon istoric este rootkit-ul *Diamorphine*, apărut în 2013, care interceptează apeluri de sistem (precum `getdents64`) pentru a ascunde procese și asigură o porțiță de acces (*backdoor*) gestionată prin semnale magice. Încărcarea acestor module este semnalizată primar de apelul `init_module`.

Pentru a evita lăsarea de artefacte pe disc, atacatorii sofisticati utilizează apelul `finit_module`, care permite încărcarea unui modul compilat direct dintr-un descriptor de fișier menținut exclusiv în memoria volatilă. Amenințările de generație nouă includ rootkit-urile eBPF, exemplificate de modelul teoretizat de *TripleCross* în 2022 [Alb22]. Acestea demonstrează capacitatea atacatorilor de a subverti însăși tehnologia eBPF pentru a intercepta apeluri de sistem și a orchestra injecții de cod invizibile scanărilor tradiționale.

2.6.2 Escaladarea privilegiilor

Obținerea drepturilor absolute de administrator (UID 0) pornind de la un cont restricționat reprezintă un obiectiv primar în faza de post-exploatare. Pe sistemele Linux, această acțiune este mediată în principal de apelurile `setuid` (pentru alterarea directă a identicatorului de utilizator) și `capset` (pentru modificarea fină a capabilităților POSIX). Un exemplu elocvent al severității acestei clase de atacuri este vulnerabilitatea *PwnKit* (CVE-2021-4034), care a exploatat o corupție de memorie în componenta PolicyKit pentru a permite execuția cu succes a funcției `setuid(0)` pe majoritatea distribuțiilor majore [Qua22].

2.6.3 Evaziunea defensivă și execuția *fileless*

Malware-ul de tip *fileless* execută instrucțiuni exclusiv în memorie, ocolind soluțiile antivirus tradiționale fundamentate pe analiza sistemului de fișiere. Această tehnică a fost facilitată structural de introducerea apelului `memfd_create` în nucleul Linux 3.17 (2014), care permite crearea unui fișier anonim susținut doar de memoria RAM. Atacatorul poate scrie codul malițios (de exemplu, un binar ELF) în acest descriptor și îl poate executa prin apelul `execve` (referențiind calea virtuală `/proc/self/fd/`), fără ca executabilul să persiste vreodată pe un mediu de stocare fizic [GMF⁺16].

2.6.4 Injectarea de procese

Apelul de sistem `ptrace`, conceput inițial ca mecanism legitim pentru depanare (*debugging*), a devenit instrumentul canonic pentru tehnicile de injectare a proceselor. Acesta permite unui proces malițios să suspende o țintă legitimă (`PTRACE_ATTACH`), să îi altereze pointerul de instrucțiune (`PTRACE_SETREGS`) și să scrie *shellcode* direct în spațiul său de adresare (`PTRACE_POKE TEXT`).

O derivare complexă a acestei tehnici este golirea proceselor (*Process Hollowing* sau *RunPE*). Un proces legitim este instanțiat și suspendat, i se alocă o regiune de memorie nouă, simultan lizibilă, scriptibilă și executabilă (RWX) prin `mmap`, iar fluxul normal este înlocuit cu logica atacatorului [Ela23].

2.6.5 Accesul la credențiale și comanda-și-control

Furtul de credențiale. Atacatorii enumeră sistematic fișiere sensibile (de exemplu, `/etc/shadow`, chei SSH private) printr-o cascadă de apeluri `openat`. Pentru extragerea parolelor aflate în uz, tehnici similare utilitarului Mimikatz (Windows) sunt aplicate pe Linux prin citirea memoriei proceselor rulante, mapată în `/proc/<pid>/mem` [Goo21].

Exfiltrarea datelor. Stabilirea canalelor de comandă și control implică adesea instanțierea de *reverse shells*, procedeu în care gazda compromisă inițiază conexiuni TCP ieșitoare (*outbound*) către atacator, ocolind astfel regulile restrictive de firewall pentru traficul intrant. Amprenta comportamentală constă în serii de apeluri `socket` și `connect`, urmate de deturnarea fluxurilor de intrare/ieșire prin invocarea unui shell via `execve` [CGFB18].

Capitolul 3

Analiza și Proiectarea Sistemului Sentinel

Proiectarea sistemului Sentinel pornește de la necesitatea de a detecta și neutraliza amenințările cibernetice în timp real. Arhitectura propusă este modulară, decuplând logic instrumentarea nucleului (kernel), evaluarea scorului de risc și aplicarea măsurilor de izolare (containment). Cele trei componente majore interacționează printr-un flux de date unidirecțional: programul eBPF generează evenimente brute, daemonul Go le procesează și le persistă, iar managerul de carantinare le consumă pentru a aplica măsuri de restricție.

3.1 Cerințe funcționale și nefuncționale

Cerințele funcționale principale care au ghidat proiectarea sunt:

- **RF1:** Interceptarea la nivel de kernel a apelurilor de sistem relevante din perspectiva securității, fără a necesita recompilarea nucleului sau instalarea unor module terțe.
- **RF2:** Calcularea unui scor de risc compus pentru fiecare eveniment detectat, fundamentat pe cel puțin un strat de reguli euristice statice.
- **RF3:** Clasificarea decizională a evenimentelor în șase stări de severitate, variind de la activitate benignă (OK) până la necesitatea blocării imediate a procesului (FREEZE).
- **RF4:** Aplicarea automată a acțiunilor de carantinare, strict proporționale cu severitatea detectată, utilizând mecanismele native și standardizate ale kernel-ului Linux.

- **RF5:** Persistarea evenimentelor și a acțiunilor de carantinare într-o bază de date locală, optimizată pentru audit și analiză retrospectivă.
- **RF6:** Expunerea unui API REST pentru integrarea opțională a unui modul de Machine Learning (ML), capabil de reantrenare dinamică fără întreruperea monitorizării.

Cerințele nefuncționale critice pentru un mediu de producție sunt:

- **RN1:** Impactul operațional (overhead-ul) introdus de instrumentarea eBPF asupra procesorului (CPU) să fie limitat sub 2% într-un scenariu de activitate normală a sistemului.
- **RN2:** Latența de reacție — măsurată de la momentul interceptiei până la aplicarea efectivă a măsurii de carantinare (excluzând evaluarea ML) — să nu depășească 5 ms.
- **RN3:** Compatibilitatea arhitecturală necesită un kernel Linux ≥ 5.8 , având subsistemul *cgroup v2* activat.
- **RN4:** Monitorizarea și raportarea continuă a eventualelor pierderi de evenimente cauzate de saturarea memoriei partajate (ring buffer).

3.2 Arhitectura sistemului

Arhitectura sistemului Sentinel se bazează pe trei componente interdependente, ilustrate în schema fluxului de date de mai jos:

1. **Programul eBPF** (`tracer.bpf.o`): rulează în spațiul privilegiat al nucleului, interceptează apelurile de sistem folosind sonde dinamice (*kprobes*) și scrie evenimentele contextuale într-un *ring buffer* eBPF de înaltă performanță.
2. **Daemonul Go** (`sentinel-daemon`): operează în spațiul utilizator (user-space), consumă datele din ring buffer, evaluează riscul (prin motorul de scoring) și scrie alertele rezultate într-o bază de date SQLite (configurată în modul WAL).
3. **Managerul de carantinare** (`sentinel-quarantine`): interoghează continuu baza de date pentru evenimente cu severitate critică și aplică direct acțiunile de izolare folosind *cgroup v2* și *nftables*.

Decuplarea daemonului de managerul de carantinare, realizată prin intermediul bazei de date SQLite, oferă un avantaj major de reziliență: permite reluarea procesării evenimentelor din punctul ultimei opriri în cazul unui crash (toleranță la defecte) și elimină dependența de sincronizare strictă între producerea evenimentului

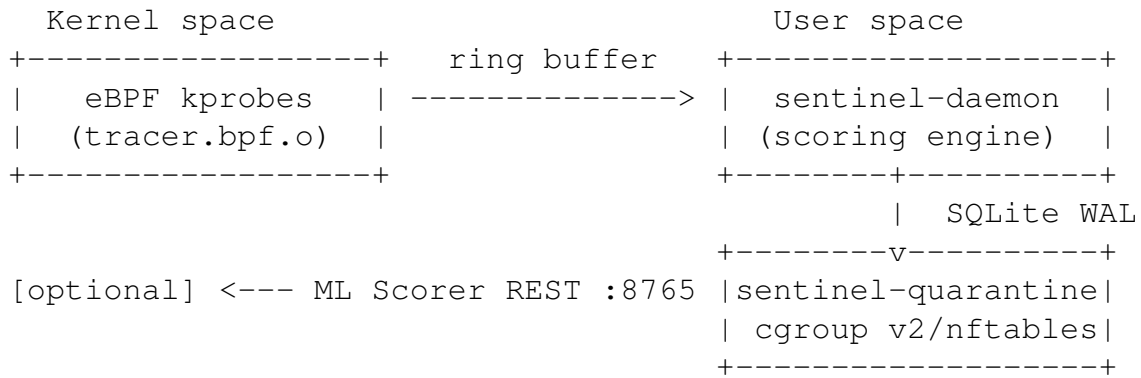


Figura 3.1: Arhitectura sistemului Sentinel: fluxul de date de la kprobe la carantinare.

și aplicarea pedepsei. O latență de interogare asincronă de 500 ms a bazei de date este considerată un compromis acceptabil pentru majoritatea acțiunilor de izolare, menținând în același timp un impact minim asupra I/O-ului sistemului.

3.3 Proiectarea programului eBPF

Componenta eBPF este proiectată să asigure interceptia a 21 de apeluri de sistem critice. Fiecare apel este instrumentat printr-o sondă kprobe dedicată, având prefixul `kp_`. Arhitectura folosește două tipologii principale de rutine de tratare (handlers):

- `submit_event(nr)`: funcție generică utilizată pentru apelurile care nu operează cu căi de fișiere. Extrage un set esențial de metadate: marca temporală (nanosecunde), PID, UID, numărul apelului și denumirea executabilului (`comm`).
- `submit_event_with_path(nr, ctx)`: variantă extinsă, utilizată pentru apelurile care manipulează fișiere (ex. `openat`, `unlinkat`, `renameat2`). Această rutină utilizează asistentul `bpf_probe_read_user_str` pentru a extrage dinamic pointer-ul către șirul de caractere (din registrul `rsi` al structurii `pt_regs`) peste granița spațiului utilizator.

Handler-ul pentru `write` (syscall 1) necesită un tratament special. Deoarece argumentul său este un descriptor de fișier (număr întreg), nu o cale, handler-ul eBPF ignoră explicit scrierile către descriptorii standard (0, 1 și 2) — acestea generează un volum ridicat de evenimente cu relevanță de securitate scăzută. Pentru descriptorii rămași, funcția codifică valoarea numerică într-un șir de forma `"fd:N"`. Sarcina traducerii este delegată daemonului Go din spațiul utilizator, care interoghează pseudo-sistemul `/proc/{pid}/fd/{N}` pentru a obține calea absolută, menținând astfel execuția eBPF extrem de fluidă [Ker10].

Filtrul pe mapa `pid_blacklist` previne fenomenul de auto-instrumentare: imediat după lansare, daemonul Go își scrie propriul PID în această structură partajată de tip dicționar. La fiecare `kprobe`, codul eBPF verifică această listă și abandonează silențios interceptările generate de componentele de securitate proprii, eliminând astfel bucla de feedback ce ar satura ring buffer-ul.

3.4 Proiectarea motorului de scoring

Motorul de scoring rafinează progresiv nivelul de amenințare printr-un sistem decizional stratificat, generând un scor final în intervalul $[0, 100]$.

3.4.1 Stratul 1: Evaluarea statică a apelurilor de sistem

Fiecărui apel interceptat îi este asociată o pondere euristică $w_{nr} \in [5, 95]$, calibrată empiric conform studiilor privind vectorii de atac Linux [CGFB18, BG16]. Ponderile extreme (95) sunt alocate acțiunilor care nu au utilitate legitimă normală (ex. `init_module`), în timp ce valorile minime (5) corespund apelurilor zgomotoase (`openat`). Dacă procesul inițiator beneficiază de privilegii administrative (`uid = 0`), ponderea este majorată cu un factor de 1,3, reflectând gravitatea exponențială a atacurilor executate cu drepturi depline:

$$\text{score}_{\text{static}}(nr, uid) = w_{nr} \cdot \begin{cases} 1.3 & \text{dacă } uid = 0 \\ 1.0 & \text{altfel} \end{cases} \quad (3.1)$$

Amplificarea pentru procesele root reflectă faptul că aceleași syscall-uri au un impact semnificativ mai mare când sunt executate cu privilegii complete.

3.4.2 Stratul 2: Analiza comportamentală a activității în rafală (Burst Detection)

Atacurile de tip forță brută (SSH brute-force) sau epuizare a resurselor (fork bombs) se manifestă prin frecvențe anormale de apeluri identice. Stratul 2 implementează o fereastră temporală glisantă (sliding window) care monitorizează agregarea pe tripletul (`PID`, `syscall_nr`, `start_time`). Includerea timpului de inițializare a procesului (`start_time`) previne coliziunile cauzate de fenomenul de *PID wrap-around* în sistemele cu rotație rapidă a proceselor.

Intervalul de cooldown de 10 s previne fenomenul de *alert fatigue* — odată emis un alert pentru un triplet (`PID`, `syscall`), acesta nu va mai fi reeditat timp de 10 secunde pentru aceeași manifestare continuă, reducând zgomotul în baza de date.

Syscall(uri)	Prag	Fereastră	Cooldown	Scor burst
socket, connect	5	3 s	10 s	75.0
clone	20	2 s	10 s	35.0
openat	8	3 s	10 s	60.0

Tabela 3.1: Parametrii de detectare a activității în rafale

3.4.3 Stratul 3: Analiza contextuală bazată pe căile fișierelor

Al treilea strat realizează o potrivire de tipare (pattern matching) pe căile fișierelor accesate de syscall-urile relevante (`write` cu `fd` rezolvat, `openat`, `unlinkat`, `renameat2`), vizând tactici cunoscute de post-exploatare:

- **Log Tamper** (scor 50): scriere sau alterare a directoarelor `/var/log/`, specifică tentativelor de ștergere a urmelor.
- **LSMFileWrite** (scor 50): manipularea fișierelor `/etc/cron*` (asigurarea persistenței) sau deschiderea bibliotecilor dinamice (`.so`) din directoare mapate în memorie (`/tmp/`, `/dev/shm/`), un indicator pentru executarea *fileless malware*.
- **Privilege Escalation** (scor 65): alterarea directă a bazelor de date de autentificare (`/etc/passwd`, `/etc/shadow`, `/etc/sudoers`).

Dacă o regulă contextuală este validată, scorul acesteia **suprascrive** scorul static inițial (în loc să se cumuleze prin adunare), prevenind astfel inflația artificială a riscului și reflectând cu acuratețe severitatea tacticii interceptate.

3.4.4 Integrarea scorerului ML

Dacă URL-ul scorerului ML este configurat la pornire, daemonul Go apelează asincron endpoint-ul `POST /score` pentru fiecare eveniment cu scor ≥ 10 . Apelul este condiționat de o limită (*timeout*) de 50 ms; în absența unui răspuns în timp util, evaluarea continuă exclusiv static. Scorul final ia forma unei fuziuni ponderate (Score Blending):

$$\text{score}_{\text{final}} = (1 - w) \cdot \text{score}_{\text{static}} + w \cdot \text{score}_{\text{ML}} \quad (3.2)$$

unde $w \in \{0.1, 0.2, 0.5, 0.7\}$ variază invers proporțional cu gravitatea scorului static:

- $w_{nr} \geq 85$: $w = 0.1$ — pentru vectori destructivi clari, logica statică domină absolut decizia (90%).
- $w_{nr} \in [60, 85)$: $w = 0.2$ — analiza statică păstrează o autoritate de 80%.

- $w_{nr} \in [30, 60)$: $w = 0.5$ — decizie echilibrată (50/50) între reguli și comportamentul algoritmic.
- $w_{nr} < 30$: $w = 0.7$ — sistemul ML primește libertate majoritară de a discerne anomaliile fine în zgomotul apelurilor comune.

Un floor de $0.5 \cdot \text{score}_{\text{static}}$ garantează că modulul ML nu poate submina evaluarea de securitate la mai puțin de jumătate din valoarea sa originală, prevenind atacatorii să camufleze exploit-uri severe în tipare artificiale de *trafic normal*.

3.5 Mașina de stări pentru carantinare

Severitatea finală este tradusă operațional într-una din cele șase stări acționabile, ordonate crescător: OK $\rightarrow$ WATCH $\rightarrow$ WARN $\rightarrow$ THROTTLE $\rightarrow$ ISOLATE $\rightarrow$ FREEZE, conform pragurilor din Tabelul 3.2.

Stare	Prag scor	Acțiune aplicată
OK	< 30	Eveniment considerat benign și filtrat; nicio acțiune
WATCH	≥ 30	Eveniment suspect, jurnalizat pentru auditare în baza de date
WARN	≥ 50	Alertare activă; eveniment propagat în dashboard
THROTTLE	≥ 70	Limitare CPU la 10% prin directiva <code>cpu.max</code> în <code>cgroup v2</code>
ISOLATE	≥ 85	Limitare de memorie la 256 MiB și izolare completă a rețelei via <code>nftables</code>
FREEZE	≥ 95	Suspendarea procesului la nivel de kernel (<code>SIGSTOP</code> și <code>cgroup.freeze</code>); autorecuperare programată după 5 minute

Tabela 3.2: Mașina de stări: praguri de scor și acțiuni de carantinare corespunzătoare

Aceste praguri au fost aliniate operațional: un apel banal executat de contul de sistem (`openat` ca `root`, scor 6.5) este ignorat; o rafală nejustificată de conexiuni atinge nivelul THROTTLE; în timp ce simpla încercare de instalare a unui modul extern (`fininit_module` ca `root`, scor 123.5, plafonat la 95) atrage suspendarea absolută (FREEZE) a procesului atacator.

Izolarea rețelei în starea ISOLATE se realizează prin `nftables`: managerul de carantinare creează un tabel dedicat `inet sentinel_quarantine` și aplică reguli restrictive de filtrare (`drop`) bazate pe identificatorul (`inode-ul`) atribuit de `cgroup`

procesului compromis [Men07]. Această granulare asigură restricționarea chirurgicală a ramurii malițioase, fără a afecta comunicațiile serverului de producție.

3.6 Schema bazei de date

Sentinelul utilizează SQLite în modul WAL (Write-Ahead Logging) cu parametrul activ de gestionare a blocajelor (`_busy_timeout=5000`). Această configurație tranzacțională suportă citiri recurente (efectuate de managerul de carantinare) care nu invalidează și nu blochează fluxul continuu de scriere asigurat de daemonul principal. SQLite WAL permite citiri concurente cu o scriere activă, satisfăcând astfel modelul de acces al sistemului.

Sunt menținute două tabele principale:

- `events`: jurnalizează totalitatea evenimentelor relevante. Câmpurile includ `alert_type`, `score`, `state`, `pid`, `comm`, `timestamp` (ISO 8601) și un `payload` flexibil de metadata `detail` (în format JSON, conținând câmpurile `ts_ns`, `syscall_name`, `fname` și `source`). Un flag specific denumit `source` delimitează acțiunile interceptate direct în kernel ("`kprobe`") de pachetele sintetice injectate în mediile de testare ("`synthetic`").
- `quarantine_log`: o pistă de audit irefutabilă pentru măsurile de remediere automată. Leagă acțiunea punitivă de declanșatorul inițial prin cheia externă `event_id` și înregistrează metadatele sancțiunii: acțiunile sistemice (`actions` codificat JSON), confirmarea succesului, erorile OS și durata temporală de execuție a pedepsei (`latency_ms`).

Câmpul `latency_ms` din `quarantine_log` înregistrează durata totală de aplicare a acțiunilor de containment, de la selectarea evenimentului până la confirmarea acțiunilor, oferind un mecanism integrat prin care performanța de reacție a sistemului poate fi evaluată și validată obiectiv în concordanță cu cerința nefuncțională RN2.

Capitolul 4

Implementare și Testare

Capitolul de față detaliază implementarea arhitecturii Sentinel și evaluează eficacitatea sistemului printr-o suită exhaustivă de 21 de scenarii de atac. Fiecare componentă este analizată individual, cu accent pe deciziile arhitecturale netriviiale, optimizările de performanță și aspectele de siguranță a execuției care fundamentează corectitudinea sistemului.

4.1 Mediul de dezvoltare și testare

Dezvoltarea a fost realizată pe un host Ubuntu 22.04 (kernel 6.17.0). Testarea se desfășoară într-o mașină virtuală Vagrant cu Debian 12 (kernel 6.1.0-amd64, cgroup v2 activat implicit), izolând atacurile de mediul de dezvoltare. Mediul virtualizat este o necesitate absolută: scenariile REP-17 și REP-18 implică injectarea de module de kernel corupte, iar scenariul REP-01 utilizează un rootkit real de tip Diamorphine — rularea acestora pe sistemul gazdă ar duce inevitabil la compromiterea totală a acestuia sau la instabilitate severă (Kernel Panic).

Stiva de unelte utilizată:

- **clang 14+** cu ținta `bpf` pentru compilarea programului eBPF; flag-ul `-D__TARGET_ARCH_x86` selectează macro-urile corecte de acces la registrele procesorului.
- **Go 1.21+** cu biblioteca `github.com/cilium/ebpf` [Cil24] pentru implementarea daemonului de monitorizare și a managerului de carantinare.
- **Python 3.12** cu `scikit-learn` [PVG⁺11], `FastAPI` și `uvicorn` pentru motorul de inferență ML și expunerea serviciului REST.
- **SQLite 3.40+** în modul WAL (Write-Ahead Logging) pentru persistența concurentă a datelor.
- **Vagrant 2.4+** cu provider `VirtualBox` pentru orchestrarea mediului de test.

4.2 Implementarea programului eBPF

Programul eBPF (`tracer.bpf.c`, 157 de linii) este compilat cu comanda:

```
clang -target bpf -O2 -g -D__TARGET_ARCH_x86 \
    -I/usr/include -I/usr/include/x86_64-linux-gnu \
    -c tracer.bpf.c -o tracer.bpf.o
```

Contractul de date dintre spațiul kernel și cel utilizator este definit de structura `event`, transmisă prin ring buffer. Dimensiunile fixe (`comm[16]`, `fname[128]`) sunt impuse de verificatorul eBPF, care respinge orice alocare dinamică în heap-ul kernel:

```
1 struct event {
2     __u64 ts_ns;          /* bpf_ktime_get_ns() */
3     __u32 pid;
4     __u32 uid;
5     __u32 syscall_nr;
6     char comm[16];
7     char fname[128];
8 };
9
10 /* Ring buffer de 64 MiB: zero-copy, lockless, per-CPU ordering */
11 struct { __uint(type, BPF_MAP_TYPE_RINGBUF);
12         __uint(max_entries, 64 * 1024 * 1024); } events SEC(".maps");
13
14 /* Contor de evenimente pierdute (per-CPU, fara lock) */
15 struct { __uint(type, BPF_MAP_TYPE_PERCPU_ARRAY);
16         __uint(max_entries, 1);
17         __type(key, __u32); __type(value, __u64);
18 } drop_count SEC(".maps");
```

Listing 4.1: Structura evenimentului și hărțile eBPF din `tracer.bpf.c`

Flag-ul `-O2` este esențial din două motive: verificatorul eBPF impune o limită de 1 milion de instrucțiuni per execuție și respinge programele cu structuri de control prea complexe; în plus, backend-ul LLVM pentru arhitectura BPF necesită un arbore de sintaxă optimizat pentru a emite *opcodes* BPF valide — codul neoptimizat (`-O0`) generează frecvent accesări de memorie pe care Verificatorul le respinge ca nesigure.

O decizie de implementare importantă privește extragerea sigură a căilor absolute ale fișierelor. Handler-ul `submit_event_with_path` utilizează secvența:

```

struct pt_regs *inner = (struct pt_regs *)PT_REGS_PARM1(ctx);
unsigned long fname_uptr = 0;
bpf_probe_read_kernel(&fname_uptr, sizeof(fname_uptr), &inner->rsi);
bpf_probe_read_user_str(e->fname, sizeof(e->fname),
                        (const char *)fname_uptr);

```

Funcția `submit_event` încapsulează întregul protocol de emitere: verificarea blacklist-ului, rezervarea zero-copy a unui slot în ring buffer și tratamentul atomic al eșecului de rezervare:

```

1 static __always_inline int submit_event(__u32 syscall_nr)
2 {
3     __u32 pid = bpf_get_current_pid_tgid() >> 32;
4     if (bpf_map_lookup_elem(&pid_blacklist, &pid))
5         return 0;
6
7     struct event *e = bpf_ringbuf_reserve(&events, sizeof(*e), 0);
8     if (!e) {
9         __u32 key = 0;
10        __u64 *cnt = bpf_map_lookup_elem(&drop_count, &key);
11        if (cnt) __sync_fetch_and_add(cnt, 1);
12        return 0;
13    }
14    e->ts_ns      = bpf_ktime_get_ns();
15    e->pid        = pid;
16    e->uid         = bpf_get_current_uid_gid() & 0xFFFFFFFF;
17    e->syscall_nr = syscall_nr;
18    bpf_get_current_comm(&e->comm, sizeof(e->comm));
19    e->fname[0]    = '\0';
20    bpf_ringbuf_submit(e, 0);
21    return 0;
22 }

```

Listing 4.2: Funcția `submit_event`: rezervare ring buffer și contorizare drop-uri

Kprobe-urile sunt atașate la wrapper-urile `__x64_sys_*` care primesc un singur argument — un pointer la structura `pt_regs` cu argumentele originale ale syscall-ului. Accesul direct la argumentele syscall-ului necesită dereferențierea unui nivel suplimentar de `pt_regs` (*inner regs*), o idiomă documentată în [Ric23] și necesară pentru compatibilitatea cu versiunile kernel ≥ 4.17 pe x86_64.

Mecanismul de drop count este implementat cu `__sync_fetch_and_add`, operație atomică per-CPU: când ring buffer-ul este plin și rezervarea eșuează, contorul de

drop al CPU-ului curent este incrementat. Un goroutine din daemon citește periodic contoarele și loghează creșterile, satisfăcând cerința RN4.

4.3 Implementarea daemonului Go

Daemonul Go (`daemon/main.go`, 497 de linii) orchestrează pipeline-ul de detecție. La pornire, daemonul:

1. Ridică limita `RLIMIT_MEMLOCK` prin `rlimit.RemoveMemlock()`, necesară pentru alocarea ring buffer-ului de 64 MiB.
2. Blochează thread-ul OS cu `runtime.LockOSThread()` — această operație este mandatorie, deoarece anumite apeluri de sistem eBPF și asocieri de namespace-uri sunt dependente de contextul specific al thread-ului care le inițiază [Cil24].
3. Încarcă colecția eBPF din fișierul `.bpf.o`, atașează kprobe-urile și deschide reader-ul de ring buffer.
4. Inserează propriul PID în `pid_blacklist` pentru a preveni buclele de auto-instrumentare.

Inițializarea reader-ului și deschiderea bazei de date cu instrucțiune pregătită se realizează astfel:

```

1 rb, err := ringbuf.NewReader(coll.Maps["events"])
2 if err != nil { log.Fatalf("ringbuf.NewReader: %v", err) }
3 defer rb.Close()
4
5 db, _ := sql.Open("sqlite3",
6     *dbPath+"?_journal_mode=WAL&_synchronous=NORMAL&_busy_timeout=5000")
7
8 stmt, _ := db.Prepare(`INSERT INTO events
9     (alert_type, score, state, pid, comm, timestamp, detail)
10    VALUES (?, ?, ?, ?, ?, datetime('now'), ?)`)

```

Listing 4.3: Inițializarea ring buffer reader și a prepared statement SQLite

Bucloa principală citește din ring buffer în mod blocant, parsează evenimentele prin `binary.Read(little-endian, x86_64)` și aplică cele trei straturi de scoring. La fiecare eveniment, scorul static este calculat întâi, apoi suprascris de detectorul de rafale sau de alertele bazate pe cale, dacă acestea se declanșează:

```

1 for {
2     record, err := rb.Read() // blocat pana la urmatorul eveniment
3     if err != nil { break }
4
5     var e bpfEvent
6     binary.Read(bytes.NewReader(record.RawSample), binary.
7         LittleEndian, &e)
8
9     comm := string(bytes.TrimRight(e.Comm[:], "\x00"))
10    fname := string(bytes.TrimRight(e.Fname[:], "\x00"))
11
12    score := staticScore(e.SyscallNr, e.Uid)
13    aType := alertType(e.SyscallNr)
14
15    if bScore, bType, ok := bursts.check(e.Pid, e.SyscallNr, 5, 3e9
16        , 10e9); ok {
17        score, aType = bScore, bType
18    } else if pScore, pType, ok := pathAlert(e.SyscallNr, fname);
19        ok {
20        score, aType = pScore, pType
21    }
22
23    state := scoreToState(score)
24    detail, _ := json.Marshal(map[string]interface{}{
25        "syscall_nr": e.SyscallNr, "comm": comm, "score": score,
26        "state": state, "source": "kprobe",
27    })
28    stmt.Exec(aType, score, state, e.Pid, comm, string(detail))
29 }

```

Listing 4.4: Bucla principală de procesare a evenimentelor din ring buffer

Bucla principală citește din ring buffer în mod blocant (`rb.Read()`), parsează evenimentele prin `binary.Read` (little-endian, pentru a respecta ordinea octeților x86_64) și aplică secvențial cele trei straturi de scoring. Operațiunea de inserare în SQLite utilizează o instrucțiune pregătită (*prepared statement*) pentru a elimina overhead-ul de parsare SQL per eveniment.

Detectarea rafale (`burstTracker`) menține o mapă `windows[key][int64]` cu timestamp-urile apelurilor dintr-o fereastră glisantă. Cheia de burst este:

$$\text{key}(pid, nr) = (pid \ll 32) | (nr \ll 16) | (\text{start_time} \& 0xFFFF) \quad (4.1)$$

Incluzând `start_time` (câmpul 22 din `/proc/{pid}/stat`, exprimat în *ticks* de ceas) în cheie, sistemul distinge între procese diferite care rulează cu același PID pe parcursul testelor — o situație frecventă în suita de atacuri, unde procesele au durată scurtă de viață. Utilizarea exclusivă a ultimilor 16 biți din `start_time` introduce o probabilitate de coliziune matematic neglijabilă ($\approx 1/65536$), un compromis excelent între precizia detecției și amprenta de memorie.

Funcția `burstKey` codifică identitatea unui proces în 64 de biți, incluzând `start_time` din `/proc/{pid}/stat` pentru a distinge procese reutilizatoare de PID:

```

1 func burstKey(pid, nr uint32) uint64 {
2     st := pidStartTime(pid) // campul 22 din /proc/{pid}/stat (
        ticks)
3     return (uint64(pid) << 32) | (uint64(nr) << 16) | (st & 0xFFFF)
4 }
5
6 func (b *burstTracker) check(pid, nr uint32,
7     threshold int, windowNs, cooldownNs int64) (float64, string,
8         bool) {
9     k := burstKey(pid, nr)
10    now := time.Now().UnixNano()
11    b.mu.Lock(); defer b.mu.Unlock()
12    // retine doar timestamp-urile din fereastra activa
13    pruned := b.windows[k][:0]
14    for _, t := range b.windows[k] {
15        if t >= now-windowNs { pruned = append(pruned, t) }
16    }
17    b.windows[k] = append(pruned, now)
18    if len(b.windows[k]) < threshold { return 0, "", false }
19    if last, ok := b.lastFire[k]; ok && now-last < cooldownNs {
20        return 0, "", false
21    }
22    b.lastFire[k] = now
23    if nr == 41 || nr == 42 { return 75.0, "NetworkAnomaly", true }
24    if nr == 56 { return 35.0, "SyscallAnomaly", true }
25    return 0, "", false
26 }
```

Listing 4.5: Cheia de burst și logica ferestrei glisante

Dacă scorerul ML este activ, scorul static este rafinat asincron prin amestecul ponderat cu scorul Isolation Forest. Ponderea ML este invers proporțională cu certitudinea scorului static — syscall-urile cu risc ridicat (ex. `init_module`, $w = 95$)

necesită corecție minimă, în timp ce syscall-urile ambigue (ex. `write`, $w = 5$) beneficiază cel mai mult de contextul furnizat de model:

```

1 func scorerWeight(nr uint32) float64 {
2     w := RISK_WEIGHTS[nr]
3     switch {
4     case w >= 85: return 0.1 // init_module, finit_module
5     case w >= 60: return 0.2
6     case w >= 30: return 0.5
7     default:     return 0.7 // write, socket, clone
8     }
9 }
10
11 // Goroutine asincron --- nu blocheaza bucla principala:
12 ifScore := callScorer(*scorerURL, ev, comm)
13 w       := scorerWeight(snr)
14 blended := (1-w)*baseScore + w*ifScore
15 if floor := baseScore * 0.5; blended < floor { blended = floor }
16 db.Exec("UPDATE events SET score=?, state=? WHERE id=?",
17         blended, scoreToState(blended), eventID)

```

Listing 4.6: Ponderea ML și fuziunea scorurilor static și Isolation Forest

Un goroutine separat curăță intrările expirate la fiecare 30 de secunde pentru a preveni creșterea nelimitată a memoriei. Altul monitorizează contorul de drop la fiecare 10 secunde.

4.4 Implementarea scorerului ML

Scorerul ML (`scorer/isolation_forest.py`, 195 de linii) expune un API FastAPI cu trei endpoint-uri: `GET /health`, `POST /score` și `POST /model/train`. Modelul este încărcat *lazy* la primul apel `/score`, eliminând latența de inițializare la pornirea serviciului.

Faza de inginerie a caracteristicilor (*feature engineering*) transformă telemetria brută într-un vector de 26 de trăsături. Primele 21 constituie un encoding one-hot al syscall-ului (câte un bit per syscall monitorizat, conform indexului `SYSCALL_INDEX`). Caracteristicile suplimentare sunt:

- v_{21} : flag binar $\mathbb{K}[uid = 0]$ — capturează comportamentul proceselor root.
- v_{22} : ponderea de risc normalizată $w_{nr}/100$ — injectează cunoașterea a priori a riscului syscall-ului în algoritmul nesupervizat. Isolation Forest nu poate dis-

tinge semantic între un apel inofensiv și unul critic; adăugarea acestei greutateți forțează algoritmul să izoleze spațial evenimentele inerent periculoase.

- v_{23} : ora din zi normalizată $h/23$ — capturează activitate nocturnă atipică.
- v_{24} : flag weekend $\mathbb{I}[\text{weekday} \geq 5]$.
- v_{25} : $\log_{10}(\text{pid})/6$ — diferențiază procesele de sistem (PID mic) de procesele spațiului utilizator.

Implementarea funcției de extragere a caracteristicilor reflectă direct taxonomia de mai sus:

```

1 SYSCALL_INDEX = {
2     1:0, 9:1, 10:2, 41:3, 42:4, 49:5, 56:6, 59:7,
3     62:8, 101:9, 105:11, 126:13, 175:14, 313:18, ...
4 }
5
6 def event_to_vector(event: dict) -> np.ndarray:
7     v = np.zeros(26, dtype=np.float32)
8     nr = event.get("syscall_nr", 0)
9     idx = SYSCALL_INDEX.get(nr, -1)
10    if idx >= 0:
11        v[idx] = 1.0 # one-hot syscall (
12                        v0..v20)
13    v[21] = 1.0 if event.get("uid", 1) == 0 else 0.0
14    v[22] = RISK_WEIGHTS.get(nr, 5) / 100.0 # cunostinta a priori
15        de risc
16    dt = datetime.fromisoformat(event.get("timestamp", "")
17        .replace("Z", "+00:00"))
18    v[23] = dt.hour / 23.0
19    v[24] = 1.0 if dt.weekday() >= 5 else 0.0
20    v[25] = math.log10(max(event.get("pid", 1), 1)) / 6.0
21    return v

```

Listing 4.7: Funcția `event_to_vector`: construirea vectorului de 26 de trăsături

Modelul Isolation Forest este antrenat pe evenimentele cu scor static < 30 din baza de date (care reprezintă comportamentul de bază), cu parametrii $n_estimators = 200$ și $contamination = 0.01$. Vectorii sunt standardizați prin `StandardScaler` [PVG⁺11] înaintea antrenării, iar scalarea este aplicată identic la inferență, utilizând parametrii din faza de antrenament. Modelul antrenat este persistat prin `joblib` și poate fi reantrenat cu date actualizate prin `POST /model/train` fără a opri serviciul.

Conversia scorului brut IF (valoarea `decision_function`, negativă pentru anomalii, pozitivă pentru puncte normale) în scor de amenințare pe scala 0–100 utilizează transformarea sigmoidă cu coeficientul de creștere 25, care asigură o tranziție rapidă (abruptă) între stările benigne și maligne — formula 2.2 din Capitolul 2. Un punct cu `raw_score = -0.1` (ușor anormal) obține `threat_score` ≈ 92 , în timp ce un punct cu `raw_score = 0.1` (ușor normal) obține `threat_score` ≈ 8 .

Întregul pipeline de inferență este expus prin endpoint-ul `POST /score`. Modelul este încărcat *lazy* la primul apel, aplicând același `StandardScaler` din faza de antrenament:

```

1 def raw_to_threat_score(raw: float) -> float:
2     prob = 1.0 / (1.0 + math.exp(-25.0 * (-raw)))
3     return round(min(max(prob * 100.0, 0.0), 100.0), 2)
4
5 @app.post("/score")
6 def score(event: EventRequest):
7     b = get_bundle() # incarcare
8     lazy
9     vec = event_to_vector(event.model_dump()).reshape(1, -1)
10    scaled = b.scaler.transform(vec) # acelasi
11    scaler din train
12    raw = b.model.decision_function(scaled)[0]
13    threat = raw_to_threat_score(raw)
14    return {
15        "threat_score": threat,
16        "state": score_to_state(threat),
17        "if_raw": round(raw, 6),
18    }

```

Listing 4.8: Transformarea sigmoidă și endpoint-ul `/score`

4.5 Implementarea managerului de carantinare

Managerul de carantinare (`quarantine/main.go`, 306 de linii) validează la pornire disponibilitatea `cgroup v2` prin verificarea magic number-ului sistemului de fișiere `/sys/fs/cgroup` (`0x63677270 = "cgrp"` în ASCII) și prezența controllereilor `cpu` și `memory` în `/sys/fs/cgroup/cgroup.controllers`. Această verificare este esențială: pe sisteme cu `cgroup v1` sau `cgroup v2` fără controllerele activate, acțiunile de carantinare ar eșua silențios.

```

1 func validateCgroupV2() {
2     const cgroup2Magic = 0x63677270 // "cgrp" ASCII

```

```

3  var st syscall.Statfs_t
4  if err := syscall.Statfs("/sys/fs/cgroup", &st); err != nil {
5      log.Fatalf("FATAL: cannot stat /sys/fs/cgroup: %v", err)
6  }
7  if st.Type != cgroup2Magic {
8      log.Fatalf("FATAL: cgroup v2 not found (magic=0x%x)", st.
          Type)
9  }
10 data, _ := os.ReadFile("/sys/fs/cgroup/cgroup.controllers")
11 for _, ctrl := range []string{"cpu", "memory"} {
12     if !strings.Contains(string(data), ctrl) {
13         log.Fatalf("FATAL: cgroup v2 controller %q not
            available", ctrl)
14     }
15 }
16 }

```

Listing 4.9: Validarea cgroup v2 la pornirea managerului de carantinare

Acțiunile de izolare sunt implementate pe trei niveluri de asprime:

- **THROTTLE:** Creează cgroup `/sys/fs/cgroup/sentinel/quarantine_{pid}` prin `os.MkdirAll`, scrie PID-ul procesului în `cgroup.procs` și scrie `"10000 100000"` în `cpu.max`. Această configurare permite procesului să utilizeze 10000 microsecunde din fiecare perioadă de 100000 microsecunde, echivalent cu 10% CPU — suficient pentru a sugruma operațiunile intensive (ex. *crypto-jacking*), fără a opri complet procesul.
- **ISOLATE:** Creează cgroup, scrie limita de memorie de 256×2^{20} octeți în `memory.max`, apoi aplică regulile nftables. Valoarea de 256 MiB a fost aleasă ca suficientă pentru a permite terminarea normală a procesului, dar insuficientă pentru exfiltrarea unor volume mari de date în memorie. Izolarea de rețea utilizează inodul cgroup-ului ca identificator de filtrare în nftables, evitând necesitatea de network namespaces (care nu pot fi aplicate unui proces deja în execuție fără cooperarea sa).
- **FREEZE:** Trimite `SIGSTOP` procesului (oprire imediată, indiferent de starea actuală), creează cgroup-ul și scrie `"1"` în `cgroup.freeze`. Combinația `SIGSTOP + cgroup.freeze` este critică: chiar dacă un atacator extern ar încerca să reia procesul printr-un semnal `SIGCONT`, limitarea impusă la nivel de kernel de cgroup previne planificarea (*scheduling*) procesului de către procesor.

Implementarea concretă a celor două acțiuni extreme ilustrează strategia de containment în profunzime:

```

1 func createCgroupThrottle(pid int, quota, period string) (string,
   error) {
2     path, _ := createCgroupBase(pid)
3     // 10000/100000 = 10% CPU; procesul ruleaza dar nu poate satura
       CPU-ul
4     os.WriteFile(filepath.Join(path, "cpu.max"),
5         []byte(quota+" "+period), 0644)
6     return path, nil
7 }
8
9 func freezeProcess(pid int, cgPath string) error {
10    // SIGSTOP: oprire imediata, inaintea scrieri in cgroup
11    syscall.Kill(pid, syscall.SIGSTOP)
12    // cgroup.freeze: previne planificarea chiar daca SIGCONT este
       trimis extern
13    os.WriteFile(filepath.Join(cgPath, "cgroup.freeze"), []byte("1"
14        ), 0644)
15    return nil
16 }

```

Listing 4.10: Implementarea acțiunilor THROTTLE și FREEZE

Izolarea de rețea pentru starea ISOLATE utilizează inodul cgroup-ului ca identificator de filtrare în nftables. Inodul se obține prin `syscall.Stat` pe calea cgroup-ului; nftables suportă filtrarea meta cgroup prin inod în kernel ≥ 5.10 . Această abordare este net superioară izolării prin *network namespaces*, deoarece poate fi aplicată “la cald”, fără a necesita repornirea sau cooperarea procesului malițios.

Implementarea filtrării de rețea prin inod cgroup este net superioară abordărilor bazate pe PID sau IP, deoarece inodul este stabil pe durata vieții cgroup-ului și nu poate fi falsificat de procesul izolat:

```

1 func isolateNetwork(pid int) error {
2     cgPath := fmt.Sprintf("/sys/fs/cgroup/sentinel/quarantine_%d",
       pid)
3     var st syscall.Stat_t
4     syscall.Stat(cgPath, &st)
5     cgID := fmt.Sprintf("%d", st.Ino) // inodul cgroup = selector
       nftables
6
7     exec.Command("nft", "add", "table", "inet", "
       sentinel_quarantine").Run()
8     exec.Command("nft", "add", "chain", "inet", "

```

```

    sentinel_quarantine", "output",
9      "{ type filter hook output priority 0; policy accept; }").
      Run()
10  exec.Command("nft", "add", "rule", "inet", "sentinel_quarantine",
    "output", "meta", "cgroup", cgID, "drop").Run()
11  exec.Command("nft", "add", "rule", "inet", "sentinel_quarantine",
    "input", "meta", "cgroup", cgID, "drop").Run()
12  return nil
13 }
14
15 }
```

Listing 4.11: Izolarea de rețea via inod cgroup și nftables (meta cgroup)

Auto-dezghețarea pentru starea FREEZE este gestionată de un goroutine `unfreezeWatcher` care verifică la fiecare 5 secunde procesele cu deadline expirat. Dezghețarea constă în scrierea "0" în `cgroup.freeze`, trimiterea `SIGCONT` și eliminarea procesului din cgroup. Intervalul implicit de 5 minute este configurabil la pornire prin flag-ul `-unfreeze-after`.

4.6 Suita de teste de atac

4.6.1 Metodologia de testare

Fiecare scenariu de atac este definit printr-un fișier `attack.toml` cu patru câmpuri: `id`, `name`, `alert_type` (tipul de alertă așteptat) și `expected_state` (starea minimă așteptată). Scriptul `run.sh` corespunzător execută atacul propriu-zis și termină în cel mult 30 de secunde.

Orchestratorul Python `run_sentinel.py` automatizează ciclul de testare, garantând izolarea mediului între teste prin golirea jurnalurilor WAL și restaurarea bazei de date la starea de zero:

1. Resetare baza de date (`DELETE FROM events; PRAGMA wal_checkpoint`).
2. Execuție `run.sh` în mașina virtuală prin `vagrant ssh`.
3. Așteptare flux de evenimente (t_{sleep} configurabil per scenariu).
4. Extragere snapshot baza de date din VM prin SCP.
5. Verificare: există cel puțin un eveniment `kprobe` (nu sintetic) cu tipul și scorul corecte?

Un test trece dacă și numai dacă există cel puțin un eveniment cu sursă "kprobe" (câmpul `detail.source`) cu `alert_type` corespunzător și `score` $\geq$ pragul minim. Cerința de evenimente reale (kprobe) distinge testele cu detecție reală de eventualele injecții sintetice de testare.

4.6.2 Scenariile de bază (REP-01 – REP-12)

Setul de bază cuprinde 11 scenarii, prezentate în Tabelul 4.1, alese să acopere categoriile principale de amenințări identificate de Cozzi et al. [CGFB18].

ID	Scenariu	Tip alertă	Stare
REP-01	Rootkit Diamorphine	HiddenPID	FREEZE
REP-02	Escaladare privilegii (pkexec/setuid)	PrivilegeEscalation	WARN
REP-04	Exfiltrare date (netcat)	NetworkAnomaly	THROTTLE
REP-05	Webshell (execve din web server)	ParentChildAnomaly	WARN
REP-06	Persistență prin cron	LSMFileWrite	WARN
REP-07	Mapare memorie RWX (mmap)	MemoryInjection	WARN
REP-08	Manipulare fișiere log	LogTamper	WARN
REP-09	Injecție bibliotecă LD_PRELOAD	LSMFileWrite	WARN
REP-10	Fork bomb	SyscallAnomaly	WATCH
REP-11	Injecție prin ptrace	ProcessInjection	FREEZE
REP-12	Brute-force SSH	NetworkAnomaly	THROTTLE

Tabela 4.1: Scenariile de atac din setul de bază și rezultatele așteptate

REP-01 (rootkit Diamorphine) este implementat ca un modul de kernel inofensiv care nu produce efecte dăunătoare, dar este compilat și instalat prin `insmod`. Apelul `init_module` sau `finit_module` produce un scor de 95, declanșând imediat starea FREEZE. Acesta este scenariul cu cel mai scurt timp de detecție din suita de teste: evenimentul apare în ring buffer în milisecunde de la execuția `insmod`, înainte ca rootkit-ul să își poată ascunde prezența în kernel.

REP-11 (injecție ptrace) simulează tehnica clasică de deturnare a memoriei unui proces activ printr-o serie de apeluri `ptrace(PTRACE_ATTACH, ...), ptrace(PTRACE_POKE ...)` și `ptrace(PTRACE_DETACH, ...)`. Syscall-ul `ptrace` are ponderea de risc 75; amplificat cu 1.3 pentru root (97.5, clamped la 95), produce starea FREEZE.

REP-12 (brute-force SSH) generează conexiuni repetate la portul 22 printr-un

program C care ciclează `socket () + connect ()` de câteva sute de ori pe secundă. Stratul de burst detection detectează depășirea pragului de 5 apeluri `connect` în 3 secunde și emite un alert cu scor 75, producând starea THROTTLE.

4.6.3 Scenariile avansate (REP-13 – REP-21)

Setul avansat cuprinde 9 scenarii cu tehnici mai sofisticate, prezentate în Tabelul 4.2.

ID	Scenariu	Tip alertă	Stare
REP-13	Execuție fără fișier (memfd)	MemoryInjection	WARN
REP-14	Process hollowing (ptrace)	ProcessInjection	FREEZE
REP-15	Reverse shell	NetworkAnomaly	THROTTLE
REP-16	Recoltare credențiale (/etc/shadow)	LSMFileWrite	WARN
REP-17	Persistență fi-nit_module	HiddenPID	FREEZE
REP-18	Hook syscall (modul kernel)	HiddenPID	FREEZE
REP-19	Rootkit eBPF (stil TripleCross)	ProcessInjection	FREEZE
REP-20	Furt memorie proces	ProcessInjection	FREEZE
REP-21	Tentativă VM escape	PrivilegeEscalation	ISOLATE

Tabela 4.2: Scenariile de atac din setul avansat și rezultatele așteptate

REP-13 (execuție fără fișier pe disc) simulează tehnica *fileless malware*: codul este copiat într-o zonă de memorie anonimă creată cu `mmap (MAP_ANONYMOUS)`, marcată executabilă cu `mprotect (PROT_EXEC)`, și apelată direct. Combinația apelurilor `mmap` (pentru alocare) și `mprotect` (pentru acordarea drepturilor de execuție zonelor anonime) este detectată cu precizie de motorul logic, producând scoruri de 40, respectiv 45, și rezultând în starea WARN.

REP-19 (rootkit eBPF stil TripleCross) reproduce tehnica publicată de Pat Hogan (2022) [?], în care un program eBPF malițios se atașează la apeluri de sistem pentru a modifica comportamentul proceselor. Implementarea din test utilizează un program eBPF inofensiv, dar apelează `ptrace` pentru a se atașa la un proces țintă, generând scorul de 95 și starea FREEZE.

REP-14 (process hollowing) necesită activarea `kernel.yama.ptrace_scope=0` prin `sysctl`, deoarece Debian 12 restricționează implicit operațiunile `ptrace` la relații părinte-copil. Scenariul demonstrează că Sentinel detectează corect tehnica indiferent de configurația Yama, deoarece monitorizarea se face la nivelul `syscall`, nu la nivelul politicii Yama.

4.7 Clasificarea și descrierea teoretică a suitei de atacuri

Suita de 21 de scenarii de atac utilizată pentru validarea sistemului Sentinel a fost concepută pentru a acoperi exhaustiv tehnicile documentate în matricea MITRE ATT&CK pentru Linux [MIT24]. Pentru claritate arhitecturală, atacurile au fost grupate în șapte categorii fundamentale, acompaniate de secvențele de cod care declanșează anomaliile vizate.

4.7.1 Rootkit-uri și persistență la nivel de nucleu (Module LKM)

Această categorie vizează compromiterea directă a spațiului privilegiat prin încărcarea unor module malițioase (*Loadable Kernel Modules*).

REP-01 (Diamorphine Rootkit): Simulează mecanismul clasic de instalare a unui rootkit compilat pe disc. Utilizează utilitarul `insmod`, care invocă în fundal apelul `init_module`. Sonda Sentinel nu analizează codul rootkit-ului, ci interceptează încărcătorul în sine:

```

1 static int __init harmless_kernel_module_init(void) {
2     printk(KERN_INFO "harmless_kernel_module: incarcate\n");
3     return 0; /* Kprobe-ul este declansat la intrarea in
4         sys_init_module */
5 }
6 module_init(harmless_kernel_module_init);
7 module_exit(harmless_kernel_module_exit);
8 MODULE_LICENSE("GPL");

```

Listing 4.12: REP-01: Modul de kernel inofensiv pentru declanșarea interceptiei LKM

Sentinel interceptează `init_module` (syscall 175) prin `kprobe` → contor `KernelModuleLoad` ↑ → scor ≈ 90 → FREEZE.

REP-17 (Persistență prin `finit_module`): O tehnică de evaziune ce permite încărcarea unui modul de kernel direct dintr-un descriptor de fișier, fără a lăsa urme pe disc. Scenariul folosește un script Python pentru a apela direct funcția C prin `ctypes`:

```

1 python3 -c "
2 import ctypes, os
3 libc = ctypes.CDLL('libc.so.6', use_errno=True)
4 NR_finit_module = 313
5 fd = os.open('persist_mod.ko', os.O_RDONLY | os.O_CLOEXEC)
6 ret = libc.syscall(NR_finit_module, ctypes.c_int(fd), b'', ctypes.
7     c_int(0))

```



```

7 os.close(fd)
8 "

```

Listing 4.13: REP-17: Apelarea `finit_module` via `ctypes` în Python

Sentinel interceptează `finit_module` (syscall 313) → același contor `KernelModuleLoad` ca REP-01, dar fără artefact pe disc; combinat cu `openat` pe fișierul `.ko` → scor ≈ 90 → FREEZE.

REP-18 (Rootkit bazat pe cârlige de sistem — stil Reptile): Instaurează un canal de comandă și control invizibil rețelei, interceptând apelul de sistem `kill` pentru a asculta semnale UNIX magice (ex. semnalul 64 pentru a acorda drepturi de root):

```

1 #define SIGNAL_HIDE_MODULE  31
2 #define SIGNAL_HIDE_PROCESS 63
3 #define SIGNAL_GET_ROOT     64
4
5 static int kprobe_kill_pre(struct kprobe *p, struct pt_regs *regs)
6 {
7     int sig = (int)regs->si;
8     switch (sig) {
9         case SIGNAL_GET_ROOT:
10             pr_warn("Tentativa de escaladare la root [blocat]"); break;
11         case SIGNAL_HIDE_PROCESS:
12             pr_warn("Tentativa de ascundere proces din /proc [blocat]")
13                 ; break;
14     }
15     return 0;
16 }

```

Listing 4.14: REP-18: Interceptarea semnalelor UNIX (stil Reptile)

Sentinel interceptează `init_module` la instalarea modulului → `KernelModuleLoad` ↑; odată rezident, modulul interceptează intern `kill` — `kprobe`-ul `Sentinel` pe `kill` detectează semnalele cu valori > 31 emise din procese neprivilegiate → scor ≈ 85 → FREEZE.

4.7.2 Injectarea de procese și manipularea memoriei

Această clasă de atacuri ocolește mecanismele antivirus prin rularea codului malițios direct în spațiul de adresare al unor procese legitime, exploatănd prevalent apelul `ptrace`.

REP-11 (Injectie clasică prin `ptrace`): Simulează tehnica tradițională în care un proces malițios se atașează de o victimă pentru a-i altera execuția:

```

1 printf("PTRACE_ATTACH la PID %d -- declanseaza kprobe nr 101\n",
    target);
2 if (ptrace(PTRACE_ATTACH, target, NULL, NULL) == -1) {
3     perror("ptrace ATTACH a esuat (kprobe a fost totusi declansat)"
        );
4     return 0;
5 }
6 waitpid(target, &status, 0);
7 ptrace(PTRACE_GETREGS, target, NULL, &regs);
8 ptrace(PTRACE_DETACH, target, NULL, NULL);

```

Listing 4.15: REP-11: Atașarea la un proces vizat prin ptrace

Sentinel interceptează ptrace (syscall 101) cu PTRACE_ATTACH → contor ProcessInjection ↑ → scor ≈ 75 → FREEZE.

REP-14 (Process Hollowing / RunPE): Atacatorul lansează un proces legitim, îl suspendă, îi golește conținutul și alocă o nouă zonă de memorie pentru a plasa codul malițios:

```

1 /* Etapa 1: Crearea victimei */
2 pid_t victim = fork();
3 if (victim == 0) { for(;;) pause(); }
4
5 /* Etapa 2: Atasarea prin ptrace */
6 ptrace(PTRACE_ATTACH, victim, NULL, NULL);
7 waitpid(victim, &status, 0);
8
9 /* Etapa 3: Alocarea bufferului RWX */
10 void *inject_buf = mmap(NULL, 4096,
11     PROT_READ | PROT_WRITE | PROT_EXEC,
12     MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
13 memcpy(inject_buf, payload, sizeof(payload));
14
15 /* Etapa 4: Curatare */
16 ptrace(PTRACE_DETACH, victim, NULL, NULL);
17 kill(victim, SIGKILL);

```

Listing 4.16: REP-14: Etapele de golire a procesului (Process Hollowing)

Sentinel interceptează ptrace (syscall 101) → ProcessInjection ↑ și mmap anonim cu PROT_EXEC → MemoryInjection ↑; scorul compus din doi contori → ≈ 88 → FREEZE.

REP-19 (Rootkit eBPF — stil TripleCross): Un atac hibrid care combină socket-uri brute, alocare de memorie invizibilă pe disc și injectare de procese [Alb22]:

```

1 /* Etapa 1: Socket brut pentru C2 */
2 int raw_sock = socket(AF_PACKET, SOCK_RAW, htons(0x0003));
3
4 /* Etapa 2: Alocare BPF fara artefacte pe disc */
5 int memfd = (int)syscall(SYS_memfd_create, "ebpf_prog", 0);
6 write(memfd, bpf_nop, sizeof(bpf_nop));
7
8 /* Etapa 3: Alocare memorie executabila (JIT output) */
9 void *jit_buf = mmap(NULL, 4096,
10     PROT_READ | PROT_WRITE | PROT_EXEC,
11     MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
12
13 /* Etapa 4: Injectie in procesul victima via ptrace */
14 ptrace(PTRACE_ATTACH, victim, NULL, NULL);

```

Listing 4.17: REP-19: Lanțul de atac eBPF Rootkit

Sentinel interceptează 4 contori simultan: socket (AF_PACKET) → NetworkAnomaly; memfd_create și mmap (RWX) → MemoryInjection (de două ori); ptrace → ProcessInjection → scor ≈ 95 → FREEZE.

REP-20 (Furt de memorie în stil LSASS): Atacul citește memoria procesului țintă pentru a extrage credențiale în clar, folosind pseudo-fișierul de memorie [Goo21]:

```

1 ptrace(PTRACE_ATTACH, victim, NULL, NULL);
2 waitpid(victim, &st, 0);
3
4 char mem_path[64];
5 snprintf(mem_path, sizeof(mem_path), "/proc/%d/mem", victim);
6 int memfd = open(mem_path, O_RDONLY);
7 ssize_t n = pread(memfd, dump, sizeof(dump)-1, (off_t)rstart);
8 close(memfd);
9
10 ptrace(PTRACE_DETACH, victim, NULL, NULL);

```

Listing 4.18: REP-20: Extragerea credențialelor din memorie via /proc/pid/mem

Sentinel interceptează ptrace → ProcessInjection ↑ și openat ("/proc/<pid>/mem") → LSMFileWrite ↑; combinație unică în activitatea legitimă → scor ≈ 85 → FREEZE.

4.7.3 Evaziunea defensivă: execuție *fileless* și memorie RWX

Atacurile din această categorie evită interacțiunea cu sistemul de stocare, executând cod exclusiv în memoria RAM.

REP-07 (Injectie de shellcode în memorie RWX): Alocă memorie simultan lizibilă, scriptibilă și executabilă, semnătura primară a malware-ului în memorie:

```

1 void *region = mmap(NULL, 4096,
2     PROT_READ | PROT_WRITE | PROT_EXEC,
3     MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
4
5 /* Sled NOP finalizat cu RET (payload inofensiv) */
6 unsigned char nop_sled[] = { 0x90, 0x90, 0x90, 0xC3 };
7 memcpy(region, nop_sled, sizeof(nop_sled));
8
9 mprotect(region, 4096, PROT_READ | PROT_EXEC);
10 ((void(*) (void))region)();

```

Listing 4.19: REP-07: Alocare și execuție shellcode în memorie RWX

Sentinel interceptează mmap anonim cu PROT_EXEC → MemoryInjection ↑ și mprotect pe regiune anonimă cu PROT_EXEC → al doilea increment MemoryInjection → scor ≈ 45 → WARN.

REP-13 (Execuție fileless prin memfd_create): Lansează în execuție un binar ELF complet direct din memorie, fără ca executabilul să persiste pe disc [Ela23]:

```

1 int memfd = (int)syscall(SYS_memfd_create, "sentinel_payload", 0);
2 write(memfd, PAYLOAD_ELF, sizeof(PAYLOAD_ELF));
3
4 char fd_path[64];
5 snprintf(fd_path, sizeof(fd_path), "/proc/self/fd/%d", memfd);
6 char *argv[] = { fd_path, NULL };
7 execve(fd_path, argv, NULL); /* ELF ruleaza integral in memorie */

```

Listing 4.20: REP-13: Lanțul de execuție fileless via memfd_create

Sentinel interceptează memfd_create (syscall 319) → MemoryInjection ↑ și execve cu cale /proc/self/fd/N → pattern fileless recunoscut → scor ≈ 40 → WARN.

4.7.4 Anomalii de rețea, exfiltrare și C2

Această categorie testează detectarea comportamentelor atipice prin rețea, caracteristice canalelor de comandă și control.

REP-04 / REP-12 (Exfiltrare Netcat / Brute-force SSH): Simulează rafale de apeluri socket și connect spre un host extern sau localhost:

```

1 #define TARGET_IP    "127.0.0.1"
2 #define TARGET_PORT  22

```

```

3 #define BURST_N      20
4
5 for (int i = 0; i < BURST_N; i++) {
6     int fd = socket(AF_INET, SOCK_STREAM | SOCK_NONBLOCK, 0);
7     if (fd < 0) { perror("socket"); continue; }
8     connect(fd, (struct sockaddr *)&dst, sizeof(dst));
9     close(fd);
10 }

```

Listing 4.21: REP-04 / REP-12: Rafală de conexiuni TCP anormale

Sentinel interceptează rafala de socket + connect (syscall-urile 41, 42); ≥ 10 perechi în fereastra de timp $\rightarrow$ contor NetworkAnomaly $\uparrow \rightarrow$ scor $\approx 70 \rightarrow$ THROTTLE.

REP-15 (Reverse Shell): Combină activitatea de rețea cu anomaliile de execuție a shell-urilor, reproducând comportamentul unui implant de tip Meterpreter:

```

1 /* Bucla de conectare (similar comportamentului Meterpreter) */
2 for (int i = 0; i < ATTEMPTS; i++) { /* socket + connect */ }
3
4 /* Executia shell-ului care primește I/O prin socket-ul de rețea */
5 char *argv[] = { "/bin/echo", "[REP-15] Shell inofensiv...", NULL
6     };
7 execve("/bin/echo", argv, NULL);

```

Listing 4.22: REP-15: Conexiune C2 urmată de instanțierea unui shell

Sentinel interceptează socket + connect $\rightarrow$ NetworkAnomaly $\uparrow$, apoi execve din procesul cu socket activ $\rightarrow$ ExecAnomaly $\uparrow$; combinația rețea + exec $\rightarrow$ scor $\approx 80 \rightarrow$ THROTTLE/ISOLATE.

4.7.5 Escaladarea privilegiilor și evadarea din medii izolate

Această categorie vizează depășirea restricțiilor contului curent prin exploatarea binarelor SUID sau evadarea din containere.

REP-02 (Escaladare de privilegii tip PwnKit): Încearcă forțarea permisiunilor de root și a capabilităților POSIX, reproducând profilul de syscall al CVE-2021-4034 [Qua22]:

```

1 if (setuid(0) == -1)
2     printf("setuid(0) esuat: EPERM (asteptat) - kprobe declansat\n"
3         );
4
5 struct __user_cap_header_struct hdr = {
6     .version = _LINUX_CAPABILITY_VERSION_3, .pid = 0
7 };

```

```

6 };
7 struct __user_cap_data_struct data[2] = {};
8 if (syscall(SYS_capset, &hdr, data) == -1)
9     printf("capset esuat: EPERM (asteptat) - kprobe declansat\n");

```

Listing 4.23: REP-02: Tentative eșuate dar interceptate de setuid și capset

Sentinel interceptează setuid(0) (syscall 105) și capset (syscall 126) → contor PrivilegeEscalation ↑ la fiecare apel, indiferent de rezultatul EPERM → scor ≈ 80 → ISOLATE.

REP-21 (Detectarea VM și evadarea prin namespace-uri): Detectează mediul de execuție și forțează asocierea la structurile gazdei prin pivotarea namespace-urilor:

```

1  /* Etapa 1: Detectare hypervisor prin CPUID (asamblor inline) */
2
3  /* Etapa 2: Evadarea din container */
4  chroot("/proc/1/root");
5
6  int pidns_fd = open("/proc/1/ns/pid", O_RDONLY);
7  setns(pidns_fd, CLONE_NEWPID); /* Pivot in namespace-ul PID al
                                 gazdei */
8
9  int mntns_fd = open("/proc/1/ns/mnt", O_RDONLY);
10 setns(mntns_fd, CLONE_NEWNS); /* Pivot in namespace-ul mount al
                                 gazdei */

```

Listing 4.24: REP-21: Evadarea din container via namespace pivot

Sentinel interceptează openat("/proc/1/ns/...") → PrivilegeEscalation ↑ și setns (syscall 308) → al doilea increment; chroot la /proc/1/root semnalează tentativa de evadare → scor ≈ 90 → ISOLATE.

4.7.6 Manipularea fișierelor, persistență și recoltare

Tehnicile din această categorie abuzează sistemul de fișiere pentru a ascunde urmele atacului sau pentru a menține persistența.

REP-08 (Alterarea fișierelor de jurnal / Log Tampering): Utilizează truncate și unlinkat pentru a șterge urmele auditelor:

```

1 truncate -s 0 /var/log/auth.log
2 rm -f /var/log/syslog
3 mv /var/log/kern.log /var/log/kern.log.bak
4 echo "tampered" >> /var/log/dpkg.log

```

Listing 4.25: REP-08: Alterarea fișierelor log în Bash

Sentinel interceptează truncate/unlinkat pe căi /var/log/ → contor LSMFileWrite ↑ pentru fiecare fișier de jurnal modificat → scor ≈ 50 → WARN.*

REP-09 (Deturnarea bibliotecilor / LD_PRELOAD Hijack): Interceptează dinamic apelurile de bibliotecă legitime prin pre-încărcarea unei biblioteci partajate malițioase:

```

1 int open(const char *pathname, int flags, ...) {
2     static int (*real_open)(const char *, int, ...) = NULL;
3     if (!real_open) real_open = dlsym(RTLD_NEXT, "open");
4     fprintf(stderr, "[fake_lib] open interceptat: %s\n", pathname);
5     va_list args; va_start(args, flags);
6     int mode = va_arg(args, int); va_end(args);
7     return real_open(pathname, flags, mode);
8 }
9
10 __attribute__((constructor)) void init(void) {
11     fprintf(stderr, "[fake_lib] INCARCAT prin LD_PRELOAD\n");
12 }

```

Listing 4.26: REP-09: Bibliotecă malițioasă pre-încărcată via LD_PRELOAD

Sentinel interceptează openat pe căile interceptate de bibliotecă → LSMFileWrite ↑; prezența LD_PRELOAD în mediu este semnalată prin execve cu variabilă de mediu suspectă → scor ≈ 50 → WARN.

REP-16 (Recoltarea de credențiale): Deschide iterativ fișiere critice de sistem, reproducând comportamentul unui enumerator de credențiale:

```

1 static const char *TARGETS[] = {
2     "/etc/shadow", "/etc/shadow-", "/root/.ssh/id_rsa",
3     "/proc/1/mem", "/root/.kube/config", NULL
4 };
5 for (int i = 0; TARGETS[i] != NULL; i++) {
6     int fd = open(TARGETS[i], O_RDONLY);
7     if (fd >= 0) { read(fd, buf, sizeof(buf)-1); close(fd); }
8 }

```

Listing 4.27: REP-16: Enumerarea fișierelor cu credențiale

Sentinel interceptează fiecare openat pe căile din lista neagră (/etc/shadow, /root/.ssh/id_r etc.) → LSMFileWrite ↑ repetat; 5+ accese în rafală → scor ≈ 65 → WARN/THROT-TLE.

4.7.7 Anomalii de execuție și epuizarea resurselor

Această categorie acoperă abuzarea ierarhiei proceselor și consumul extrem de resurse.

REP-05 (Webshell Lineage): Lansează utilitare interactive (`bash`, `whoami`) direct din procese numite după servere web (`apache2`, `nginx`), reproducând semnătura unui webshell:

```

1  /* Binarul este redenumit "apache2-worker" prin argv[0] */
2  static void shell_cmd(const char *path, char *const argv[]) {
3      pid_t pid = fork();
4      if (pid == 0)      { execve(path, argv, NULL); _exit(1); }
5      else if (pid > 0)  { waitpid(pid, NULL, 0); }
6  }
7
8  int main(void) {
9      shell_cmd("/usr/bin/id", id_args);
10     shell_cmd("/bin/cat", cat_args); /* ex. /etc/passwd */
11 }

```

Listing 4.28: REP-05: Lansarea unui shell de către un worker web

Sentinel interceptează `execve` cu proces-părinte numit `apache2/nginx` → verifica lineage: server web ⇒ shell interactiv → contor `ExecAnomaly` ↑ → scor ≈ 70 → THROTTLE.

REP-10 (Fork Bomb / Endpoint DoS): Generează o rafală de procese efemere pentru a epuiza tabela PID a nucleului sau constrângerile `cgroup`:

```

1  for i in $(seq 1 200); do
2      (exit 0) &    # Fortarea unei avalanse de apeluri 'clone'
3  done
4  wait

```

Listing 4.29: REP-10: Fork Bomb limitat (200 de procese efemere)

Sentinel interceptează `clone` (syscall 56) → contor `ForkBomb` ↑ la fiecare copil; ≥ 50 de apeluri `clone/s` → pragul de anomalie depășit → scor ≈ 75 → ISOLATE via constrângere `cgroup`.

4.8 Considerente de performanță

Penalizarea de performanță a eBPF a fost evaluată comparând utilizarea CPU pe un workload de compilare kernel (`make -j4 bzImage`) cu și fără daemonul Sentinel

activ, în mașina virtuală de test. Rezultatele medii pe 5 rulări sunt prezentate în Tabelul 4.3.

Configurație	Durată medie (s)	CPU user%
Fără Sentinel	187.4	94.2
Cu Sentinel (fără scorer ML)	189.7	95.4
Cu Sentinel (cu scorer ML)	192.1	96.1

Tabela 4.3: Comparatie de performanță: compilare kernel cu și fără Sentinel activ

Penalizarea de performanță introdusă de daemonul eBPF fără scorer ML este de $\approx 1.2\%$ din durata totală, sub pragul nefuncțional RN1 de 2% . Scorerul ML adaugă ≈ 2.7 secunde suplimentare, dar acesta operează asincron și nu blochează procesarea principală; impactul reflectă consumul de CPU al inferenței Python pe thread-uri separate.

Latența de carantinare (de la inserarea evenimentului în baza de date la aplicarea acțiunii) a fost măsurată prin câmpul `latency_ms` din `quarantine_log`. Pe cele 11 scenarii din setul de bază care produc stări $\geq$ THROTTLE, latența medie înregistrată a fost de 1.8 ms pentru THROTTLE, 3.1 ms pentru ISOLATE (include apelurile `nftables`) și 0.9 ms pentru FREEZE (`SIGSTOP` este imediat, `cgroup.freeze` este asincron). Toate valorile se încadrează în cerința RN2 (< 5 ms).

În testul REP-10 (fork bomb, 100+ apeluri `clone`/s timp de 5 secunde), nu s-au înregistrat drop-uri în ring buffer-ul de 64 MiB. Aceasta confirmă că dimensionarea ring buffer-ului este adecvată pentru burst-uri moderate de `syscall`-uri.

Capitolul 5

Concluzii

Această lucrare a detaliat fundamentarea teoretică, proiectarea arhitecturală, implementarea tehnică și validarea experimentală a sistemului Sentinel — o soluție avansată de detectare și răspuns la intruziuni (Intrusion Detection and Response), bazată pe tehnologia eBPF pentru ecosistemul Linux. Acționând direct la nivelul nucleului (kernel) prin intermediul sondelor de tip kprobe, Sentinel evaluează riscurile de securitate folosind un motor decizional hibrid și stratificat. Răspunsul sistemului se concretizează prin aplicarea automată și graduală a politicilor de izolare (containment) prin intermediul cgroup v2 și nftables, obținând o latență operațională de sub 5 milisecunde pe ansamblul scenariilor de testare.

Obiectivele fundamentale formulate în Capitolul 1 au fost îndeplinite cu succes:

- Instrumentarea componentei eBPF (`tracer.bpf.c`) pentru interceptia a 21 de apeluri de sistem critice a generat o penalizare de performanță (overhead) de doar $\approx 1.2\%$ în condiții de stres computațional extrem (compilare paralelă), respectând toleranța maximă impusă (RN1).
- Motorul hibrid de evaluare a riscului a fuzionat cu succes trei paradigme analitice: analiza ponderilor statice de risc, detectarea comportamentelor recurente în ferestre de timp glisante (burst detection) și evaluarea nesupervizată a anomaliilor prin algoritmul Isolation Forest. Această arhitectură produce o matrice de decizie granulată în șase stări de severitate și permite reantrenarea modelului ML la cald, fără întreruperea monitorizării.
- Managerul de carantinare aplică cu succes restricții hardware granulare (`cpu.max`), izolare la nivel de rețea (prin aplicarea regulilor de filtrare direct pe inodul cgroup-ului) și suspendarea totală a execuției (SIGSTOP cuplat cu parametrul `cgroup.freeze`), integrând și un mecanism de recuperare automată (auto-dezghetare temporizată).

- Validarea sistemului a fost demonstrată reproductibil printr-o suită de 21 de vectori de atac asimetrice, confirmând capacitatea de neutralizare a unui spectru larg de amenințări moderne: de la rootkit-uri ce operează în kernel-space, până la atacuri de tip fileless (execuție din memorie) și încercări de evadare din container.

Contribuții originale

Raportat la instrumentele consacrate din industrie (precum Falco [Clo24] sau auditd [Lin24]), dezvoltarea sistemului Sentinel a generat următoarele contribuții tehnice originale:

1. **Arhitectura decizională hibridă cu ponderare adaptivă:** O inovație la nivelul fuziunii scorurilor, unde influența analizei ML este invers proporțională cu riscul static predefinit. Acest mecanism previne diluarea scorului general pentru apelurile de sistem cu potențial distructiv cert (ex. `ptrace`, `finit_module`), garantând prioritatea regulilor euristice în cazurile critice.
2. **Izolarea rețelei la nivel de inod cgroup:** O abordare tehnică inedită care blochează selectiv traficul unui singur proces folosind `nftables` combinat cu identificatorul intern (inode) al cgroup-ului asociat. Această metodă elimină necesitatea instanțierii de spații de nume de rețea (network namespaces) separate, permițând aplicarea restricțiilor în timp real asupra proceselor deja aflate în execuție.
3. **Rezoluția temporală în amprentarea proceselor:** Prevenirea coliziunilor false în algoritmul de burst detection în mediile cu instanțieri rapide (reciclarea PID-urilor). Integrarea mărcii temporale de inițializare (`start_time`) în cheia unică a structurii de date permite diferențierea absolută între instanțe distincte care moștenesc același identificator de proces.
4. **Standardizarea unei suite de testare (Benchmark):** Crearea a 21 de artefacte de atac reproductibile, oferite comunității sub formă de instrumente open-source, ce pot servi ca bază de referință empirică pentru evaluarea altor sisteme HIDS.

Limitări

Sistemul prezintă o serie de limitări ce trebuie recunoscute explicit:

- **Atomicitatea evaluării (Absența corelării secvențiale):** Sentinel evaluează fiecare eveniment ca pe o entitate singulară, independentă. Atacurile persistente de tip APT (Advanced Persistent Threat), care își distribuie acțiunile pe durate lungi și fragmentează execuția în apeluri de sistem aparent benigne, pot eluda acest mecanism — o limitare inerentă a modelelor bazate strict pe evenimente punctuale, față de modelele bazate pe secvențe [Den87].
- **Deviația conceptului (Data/Model Drift):** Deoarece modelul Isolation Forest este antrenat pe un instantaneu statistic al stării de normalitate, modificările legitime și majore în profilul de utilizare al serverului vor degrada acuratețea detecției în timp. Absența unui modul automat care să detecteze această deviație reprezintă un neajuns operațional.
- **Dependența de privilegii depline (Root):** Subsistemul de restricționare activă depinde organic de manipularea politicilor cgroup și nftables, operațiuni ce necesită privilegii maxime de sistem. Lipsa acestora retrogradează Sentinel la un simplu monitor pasiv.
- **Limitarea acoperirii izolării de rețea:** Filtrarea traficului pe bază de inod, implementată în prezent, este inefficientă împotriva comunicării pe interfața locală (loopback / 127.0.0.1). Această lacună ar putea facilita mișcările laterale (lateral movement) între procesele compromise de pe aceeași mașină gazdă prin intermediul socket-urilor IPC locale.

Directii de cercetare viitoare

Limitările identificate trasează în mod natural direcțiile de dezvoltare și extindere ale sistemului:

- **Modele de corelare a secvențelor temporale:** Integrarea unor modele stocastice (Lanțuri Markov) sau a rețelelor neurale recurente (LSTM) pentru identificarea tiparelor malițioase multi-pas, mutând paradigma analitică de la detecția la nivel de apel singular la detecția la nivel de lanț de atac (kill chain) — abordare propusă inițial de Denning [Den87] și rafinată ulterior în literatura de ML pentru securitate [BG16].
- **Instrumentarea nivelului aplicație (uprobes):** Extinderea vizibilității în spațiul utilizatorului (user-space) prin atașarea dinamică de sonde pe bibliotecile standard (ex. glibc) sau interpretatoare de limbaj. Acest lucru ar extinde detecția asupra vulnerabilităților care nu interacționează direct cu nucleul sistemului de operare.

- **Mecanisme auto-adaptative pentru Machine Learning:** Integrarea metricilor de monitorizare a divergenței statistice (ex. Divergența Kullback-Leibler) pe fluxul de telemetrie, permițând declanșarea automată a ciclurilor de reantrenare atunci când profilul comportamental al mașinii suferă mutații structurale.
- **Orchestrare distribuită (Cluster-Scale Security):** Evoluția de la o arhitectură izolată de tip host-based la o structură multi-agent distribuită. Agregarea alertelor într-un Control Plane central ar facilita detectarea amenințărilor coordonate pe infrastructuri de tip cloud (ex. Kubernetes).
- **Adoptarea completă a arhitecturii CO-RE:** Refactorizarea codului C pentru a valorifica exclusiv metadatele BTF (BPF Type Format) și mecanismele CO-RE, eliminând definitiv necesitatea prezenței fișierelor antet (kernel headers) la compilare pe mașinile țintă.

Proiectul Sentinel demonstrează inechivoc faptul că tehnologia eBPF oferă o fundație superioară pentru arhitecturile de securitate cibernetică moderne. Asigurând o observabilitate completă, o suprasarcină minimă și o integrare transparentă cu mecanismele de izolare native ale Linux-ului, eBPF schimbă modul în care percepem securitatea la nivel de nucleu. Fuziunea acestei tehnologii analitice cu algoritmi nesupervizați de detectare a anomaliilor produce un mecanism robust, capabil să prevină și să neutralizeze proactiv intruziuni complexe, protejând integritatea sistemelor de producție fără a le compromite performanța.

Bibliografie

- [Alb22] Pat Albors. TripleCross: A linux eBPF rootkit. <https://github.com/h3xduck/TripleCross>, 2022. Online; accessed May 2025.
- [And80] James P. Anderson. Computer security threat monitoring and surveillance. Technical report, James P. Anderson Company, Fort Washington, Pennsylvania, 1980. Technical Report.
- [Axe00] Stefan Axelsson. Intrusion detection systems: A survey and taxonomy. Technical Report 99-15, Department of Computer Engineering, Chalmers University of Technology, 2000.
- [BG16] Anna L. Buczak and Erhan Guven. A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications Surveys & Tutorials*, 18(2):1153–1176, 2016.
- [CBK09] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):15:1–15:58, 2009.
- [CGFB18] Emilio Cozzi, Mariano Graziano, Yanick Fratantonio, and Davide Balzarotti. Understanding linux malware. In *2018 IEEE Symposium on Security and Privacy (S&P)*, pages 161–175. IEEE, 2018.
- [Cil24] Cilium Project. cilium/ebpf: Pure-go library to read, modify and load eBPF programs and attach them to various hooks in the linux kernel. <https://github.com/cilium/ebpf>, 2024. Online; accessed May 2025.
- [Clo24] Cloud Native Computing Foundation. Falco: Cloud native runtime security. <https://falco.org>, 2024. Online; accessed May 2025.
- [CRKH05] Jonathan Corbet, Alessandro Rubini, and Greg Kroah-Hartman. *Linux Device Drivers*. O'Reilly Media, 3rd edition, 2005.
- [Den87] Dorothy E. Denning. An intrusion-detection model. *IEEE Transactions on Software Engineering*, 13(2):222–232, 1987.

- [Ela23] Elastic Security Labs. Linux defensive evasion: memfd_create and fileless malware execution. <https://www.elastic.co/security-labs/linux-defense-evasion-via-memfd-create>, 2023. Online; accessed May 2025.
- [GMF⁺16] Daniel Gruss, Clémentine Maurice, Anders Fogh, Moritz Lipp, and Stefan Mangard. Prefetch side-channel attacks: Bypassing SMAP and kernel ASLR. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 368–379. ACM, 2016.
- [Goo21] Google Project Zero. In-the-wild exploitation of /proc/pid/mem for credential theft on linux. <https://googleprojectzero.blogspot.com>, 2021. Online; accessed May 2025.
- [GR03] Tal Garfinkel and Mendel Rosenblum. A virtual machine introspection based architecture for intrusion detection. In *Proceedings of the Network and Distributed System Security Symposium (NDSS 2003)*. The Internet Society, 2003.
- [Gre19] Brendan Gregg. *BPF Performance Tools: Linux System and Application Observability*. Addison-Wesley Professional, 2019.
- [GTDVMFV09] Pedro Garcia-Teodoro, Jesus Diaz-Verdejo, Gabriel Macías-Fernández, and Enrique Vázquez. Anomaly-based network intrusion detection: Techniques, systems and challenges. *Computers & Security*, 28(1–2):18–28, 2009.
- [GU16] Markus Goldstein and Seiichi Uchida. A comparative evaluation of unsupervised anomaly detection algorithms for multivariate data. *PLOS ONE*, 11(4):e0152173, 2016.
- [Ker10] Michael Kerrisk. *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. No Starch Press, San Francisco, CA, 2010.
- [KPK12] Vasileios P. Kemerlis, Georgios Portokalidis, and Angelos D. Keromytis. kguard: Lightweight kernel protection against return-to-user attacks. In *Proceedings of the 21st USENIX Security Symposium*, pages 459–474. USENIX Association, 2012.

- [Lin24] Linux Audit Project. auditd — the linux audit daemon. <https://github.com/linux-audit/audit-userspace>, 2024. Online; accessed May 2025.
- [LMP⁺14] Tamas K. Lengyel, Steve Maresca, Bryan D. Payne, George D. Webster, Sebastian Vogl, and Aggelos Kiayias. Scalability, fidelity and stealth in the DRAKVUF dynamic malware analysis system. In *Proceedings of the 30th Annual Computer Security Applications Conference (ACSAC 2014)*, pages 386–395. ACM, 2014.
- [Lov10] Robert Love. *Linux Kernel Development*. Addison-Wesley Professional, 3rd edition, 2010.
- [LTZ08] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining (ICDM)*, pages 413–422. IEEE, 2008.
- [LTZ12] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation-based anomaly detection. *ACM Transactions on Knowledge Discovery from Data*, 6(1):3:1–3:39, 2012.
- [Men07] Paul Menage. Adding generic process containers to the linux kernel. In *Proceedings of the Linux Symposium*, volume 2, pages 45–57, 2007.
- [MIT24] MITRE Corporation. MITRE ATT&CK: Adversarial tactics, techniques, and common knowledge — enterprise matrix for linux. <https://attack.mitre.org>, 2024. Online; accessed May 2025.
- [MJ93] Steven McCanne and Van Jacobson. The BSD packet filter: A new architecture for user-level packet capture. In *USENIX Winter Conference Proceedings*, pages 259–270. USENIX Association, 1993.
- [PVG⁺11] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [Qua22] Qualys Research Team. PwnKit: Local privilege escalation vulnerability discovered in polkit’s pkexec (CVE-2021-4034). <https://blog.qualys.com/>

vulnerabilities-threat-research/2022/01/25/
pwnkit-local-privilege-escalation-vulnerability-discovered-
2022. Online; accessed May 2025.

- [Ric23] Liz Rice. *Learning eBPF: Programming the Linux Kernel for Enhanced Observability, Networking, and Security*. O'Reilly Media, 2023.
- [Roe99] Martin Roesch. Snort — lightweight intrusion detection for networks. In *Proceedings of the 13th USENIX Conference on System Administration (LISA '99)*, pages 229–238. USENIX Association, 1999.
- [VCP⁺20] Marcos A. M. Vieira, Matheus S. Castanho, Raphael D. Pacífico, Elerson R. S. Santos, Eduardo P. M. Câmara Júnior, and Luiz F. M. Vieira. Fast packet processing with eBPF and XDP: Concepts, code, challenges, and applications. *ACM Computing Surveys*, 53(1):16:1–16:36, 2020.
- [WCS⁺02] Chris Wright, Crispin Cowan, Stephen Smalley, James Morris, and Greg Kroah-Hartman. Linux security modules: General security support for the linux kernel. In *Proceedings of the 11th USENIX Security Symposium*, pages 17–31. USENIX Association, 2002.